Project Report

On

# INSURANCE MANAGEMENT SYSTEM

# ACKNOWLEDGEMENT

We take this occasion to thank God, almighty for blessing us with his grace and taking our endeavor to a successful culmination. We extend our sincere and heart felt thanks to our esteemed guide, Mr. XYZ for providing us with the right guidance and advice at the crucial junctures and for showing us the right way. We extend our sincere thanks to our respected head of the division Mr.XYZ, for allowing us to use the facilities available. We would like to thank the other faculty members also, at this occasion. Last but not the least, we would like to thank friends for the support and encouragement they have given us during the course of our work.

# ABSTRACT

**PROJECT TITLE : Insurance Management System**

**APPLICATION TYPE:** Web Application

**DESCRIPTION**

The main purpose of this project is to estimate some important statistics of Micro insurance in the current market of India by gathering and analyzing data which is available. The statistics will be useful to prepare better insurance plans to boost up the Life Insurance industry in India.

**MODULES**

a) Admin module: This module allows only registered members to log in and manage the users and article.
b) users module: This module allows to known the premium details of the users and product details.

**PROGRAMMING ENVIRONMENT**

**Operating System** : Windows

**Application type** : Web Application

**Frontend** : PHP

**Database** : MySql

**Server** : Apache

# INTRODUCTION

## Purpose of the System

This web site is an online micro insurance Analysis and information management system that provides easy access of information regarding the people and resources of insurance. This site is not a static site but with wonderful dynamic facilities like search tools for insurance awareness articles, guidelines, illustrations through images for visitors. This site also provides several dynamic features.

## Scope of the Developed System

The developed system is applicable to only MIRC to provide online services to its member organizations.

**Insurance Management System**

In this project we use PHP and Mysql database. It has two module

1.User Module

2.Admin Module.

**User Module:**

**In this module there is Two Section**

1. **Dashboard**: In this section, user can see all detail in brief.

2. **Insurance** : In this section, user can apply for policy and check his policy is approve or reject(which is done by admin).

3. **Ticket**: In this section, user can raise ticket against any complain and see status of his/her ticket

User can also update his/ her profile, change password and recover password.

**Admin Module**

**In this module there is nine sections**

**1. Dashboard**: In this section admin can see all detail in brief.

**2. Insurance Category**: In this section admin can manage categories(add and update).

**3. Insurance Subcategory**: In this section admin can manage sub category(add and update).

**4. Insurance Policy**: In this section admin can manage insurance policy(add and update policy).

**5. User Detail**: In this section admin can manage all user detail.

**6. Policy Holders**: In this section admin can manage all insured policy. Admin can view policy on the basis of status(pending policy, approved policy and disapproved policy). Admin also can approve pending policy.

**7. Tickets:** In this section admin can view detail of issues raised by user and can also update remark on particular tickets.

**8. Reports**: In this section, admin generate between dates report of ticket generation and policy request.

**9. Search**: In this section, admin can search ticket and policy.

Admin can also update his profile and change his password

# SYSTEM ENVIRONMENT

## Hardware Configuration

1. Pentium IV Processor
2. 512 MB RAM
3. 40GB HDD
4. 1024 * 768 Resolution Color Monitor

**Note: This is not the "System Requirements".**

## Software Configuration

1. OS : Windows XP
2. PHP Triad (PHP5.6, MySQL, Apache, and PHPMyAdmin)

## Software Features

### PHP TRIAD

PHPTriad installs a complete working PHP/MySQL server environment on Windows platforms (9x/ NT). Installs PHP, MySQL, Apache, and PHPMyAdmin.

### PHP

PHP, which stands for Hypertext Preprocessor, is a server-side scripting language primarily designed for web development. It is an open-source, general-purpose scripting language that is especially suited for creating dynamic web pages and applications. PHP is embedded within HTML code and is executed on the server, generating dynamic content that is then sent to the client's web browser for display.

**Key features and aspects of PHP include:**

**1. Server-side scripting:** PHP is executed on the server, and the results are sent to the client's browser. This allows for the generation of dynamic content before it reaches the user.

**2. Interoperability:** PHP can work seamlessly with various databases, web servers, and protocols. It supports a wide range of databases, such as MySQL, PostgreSQL, and SQLite.

**3. Open source:** PHP is open-source software, which means that it is freely available, and developers can contribute to its development and improvement.

**4. Embedding in HTML:** PHP code is often embedded directly within HTML code, allowing developers to intersperse dynamic content generation with static HTML content.

**5. Extensive library support:** PHP has a vast standard library, offering modules and functions for a variety of tasks, from working with databases to handling HTTP requests and more.

**6. Cross-platform compatibility:** PHP can run on various operating systems, including Windows, Linux, macOS, and others.

**7. Community support:** PHP has a large and active community of developers, which provides support, tutorials, and resources for users.

PHP is commonly used in combination with a web server (such as Apache or Nginx) and a database management system (such as MySQL or PostgreSQL) to create dynamic web applications. It is a popular choice for building websites and web applications due to its ease of use, flexibility, and widespread adoption.

**Syntax**

```
<html>
```

```
<head>
    <title>PHP Test </title>
</head>
<body>
    <?php echo "<p> Hello World </p>"; ?>
</body></html>
```

Note : - Code in bold letters shows the PHP code embedded within HTML

**MySQL**

MySQL is an open-source relational database management system (RDBMS) that is widely used for managing and organizing structured data. It is a central component of the LAMP (Linux, Apache, MySQL, PHP/Python/Perl) and MEAN (MongoDB, Express.js, AngularJS, Node.js) stacks, which are popular choices for web development.

**Here are some key characteristics and features of MySQL:**

Relational Database Management System (RDBMS): MySQL follows the principles of a relational database, where data is organized into tables with rows and columns. It supports SQL (Structured Query Language) for defining, manipulating, and querying the data.

**Open Source:** MySQL is an open-source software, meaning that its source code is freely available, and users can modify and distribute it under the terms of the GNU General Public License (GPL).

**Scalability:** MySQL can handle large amounts of data and is known for its scalability. It is used in a wide range of applications, from small websites to large-scale enterprise

systems.

**Cross-Platform Compatibility:** MySQL is compatible with various operating systems, including Linux, Windows, macOS, and others. This makes it versatile and suitable for different environments.

**Performance:** MySQL is designed for high performance. It employs various optimization techniques, indexing mechanisms, and caching strategies to deliver efficient data retrieval and manipulation.

**Community Support:** Similar to PHP, MySQL has a large and active community of developers. This community provides support, documentation, and additional tools to enhance the functionality of MySQL.

**Security Features:** MySQL includes various security features, such as user authentication, encryption, and access control, to protect data stored in the database.

**Database Tables**

A database most often contains one or more tables. Each table is identified by a name (e.g. “Customers” or “Orders”). Tables contain records (rows) with data.

**Queries**

A query is a question or a request. With MySQL, we can query a database for specific information and have a record set returned.

**Create a connection to a database**

Before you can access data in a database, you must create a connection to the

database. In PHP, this is done with the mysqli_connect() function.

**Syntax**

```
<?php
$con=mysqli_connect("host-name", "db-username", "db-apassword", "db-name");
if(mysqli_connect_errno()){
echo "Connection Fail".mysqli_connect_error();
}
?>
```

**Closing a Connection**

The connection will be closed automatically when the script ends. To close the connection before, use the mysqli_close() function:

---

**Create a Database**

The CREATE DATABASE statement is used to create a database in MySQL.

**Syntax**

CREATE DATABASE database_name

To get PHP to execute the statement above we must use the mysql_query() function. This function is used to send a query or command to a MySQL connection.

**Create a Table**

The CREATE TABLE statement is used to create a table in MySQL

**Syntax**

```
CREATE TABLE table_name
                (
                  column_name1
                  data_type,
                  column_name2
                  data_type,
                  column_name3
                  data_type,
                  ....
)
```

**phpmydmin**

**phpMyAdmin** is an open source tool written in PHP intended to handle the administration of MySQL over the World Wide Web. phpMyAdmin supports a wide range of operations with MySQL. Currently it can create and drop databases, create/drop/alter tables, delete/edit/add fields, execute any SQL statement, manage users and permissions, and manage keys on fields. while you still have the ability to directly execute any SQL statement. phpMyAdmin can manage a whole MySQL server (needs a super-user) as well as a single database. To accomplish the latter you'll need a properly set up MySQL user who can read/write only the desired database. It's up to you to look up the appropriate part in the MySQL manual.

**phpMyAdmin can:**

- browse and drop databases, tables, views, fields and indexes
- create, copy, drop, rename and alter databases, tables, fields and indexes
- maintenance server, databases and tables, with proposals on server configuration
- execute, edit and bookmark any SQL-statement, even batch-queries
- load text files into tables
- create and read dumps of tables
- export data to various formats: CSV, XML, PDF, ISO/IEC 26300 - OpenDocument Text and Spreadsheet, Word, Excel and $L^{A}T_{E}X$ formats

- administer multiple servers
- manage MySQL users and privileges
- check referential integrity in MyISAM tables
- using Query-by-example (QBE), create complex queries automatically connecting required tables
- create PDF graphics of your Database layout
- search globally in a database or a subset of it
- transform stored data into any format using a set of predefined functions,

  like displaying BLOB-data as image or download-link
- support InnoDB tables and foreign keys
- support mysqli, the improved MySQL extension.

**Apache Web server**

Often referred to as simply *Apache*, a public-domain open source Web server developed by a loosely-knit group of programmers. The first version of Apache, based on the NCSA httpd Web server, was developed in 1995.

Core development of the Apache Web server is performed by a group of about 20 volunteer programmers, called the *Apache Group.* However, because the source code is freely available, anyone can adapt the server for specific needs, and there is a large public library of Apache add-ons. In many respects, development of Apache is similar to development of the Linux operating system.

The original version of Apache was written for UNIX, but there are now versions that run under OS/ 2, Windows and other platforms. The name is a tribute to the Native American Apache Indian tribe, a tribe well known for its endurance and skill in warfare. A common misunderstanding is that it was called Apache because it was developed from existing NCSA code plus various patches, hence the name *a patchy server*, or Apache server.

Apache consistently rates as the world's most popular Web server according to analyst surveys. Apache has attracted so much interest because it is full-featured, reliable, and free. Originally developed for UNIX™ operating systems, Apache has been updated to run on Windows, OS/2, and other platforms. One aspect of Apache that some site administrators find confusing — especially those unfamiliar with UNIX-style software — is its configuration scheme. Instead of using a point-and-click graphic user interface (GUI) or Windows Registry keys as most other.

**Configuration Files**

Apache uses a system of three text files for managing its configuration data. All three of these files (almost always) appear in Apache's ./conf directory and are designed to be edited by system administrators:

1. httpd.conf for general settings
2. srm.conf for resource settings
3. access.conf for security settings

When Apache first starts, these files are processed in the order shown above. Originally, the initial installation of Apache included default entries within each of the

three files. In the most recent versions of Apache, however, the default installation has changed. Now httpd.conf is treated as the “master” configuration file and it contains all of the settings. Both srm.conf and access.conf still exist in the installation, but they contain no settings and are empty except for some comments.

**Inside Httpd.conf**

Traditionaly httpd.conf contained general settings such as the ServerName and Port number. These entries appear as follows in the file: ServerName compnetworking.about.com Port 80 The term “httpd” stands for *HTTP Daemon*. Recall that in a UNIX environment, the term *daemon* refers to a type of process designed to launch at system boot and continue running for very long periods of time. This file contains a number of other entries (technically called directives), but for most of these, modifications are optional. Probably the most useful of these entries is ServerAdmin.

**Access and Security Settings**

It is recommended practice now for Apache administrators to manage their resource and security settings from httpd.conf. Administrators of older versions of Apache can simply cut their entries from srm.conf and access.conf and paste them into the master file. If an administrator wants to go one step further and delete the two empty files, they should also place the following entries in httpd.conf to prevent Apache from attempting to access them.

# SYSTEM DESIGN

## DATA DESIGN DIAGRAMS

UML is a latest concept of presenting various diagrams of a project. UML stands for Unified Modeling Language.

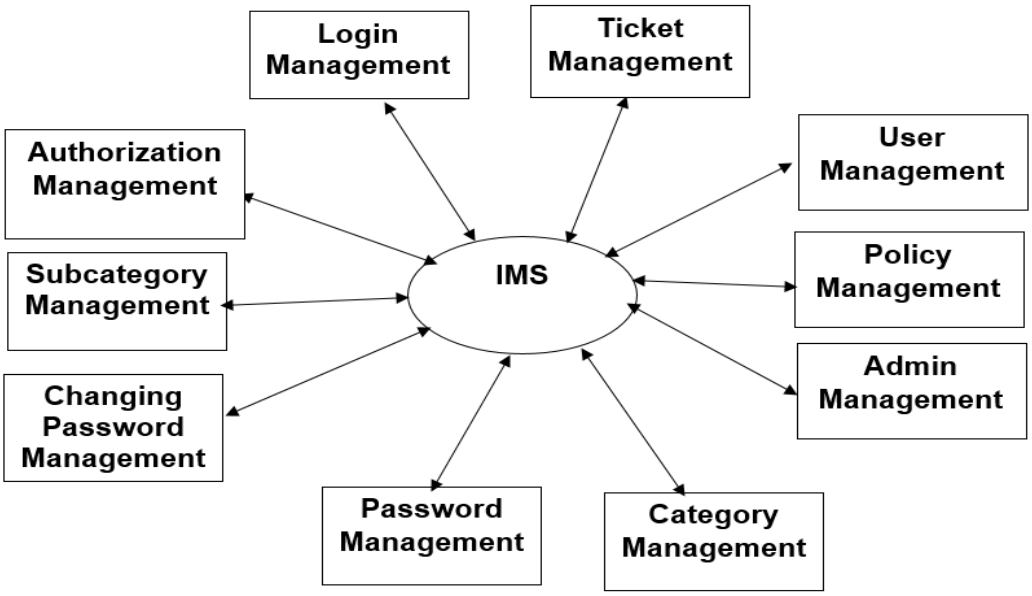
The purpose of this section is to show the following type of conceptual/logical diagrams of the proposed project.

- Data Flow diagram of the project
- Use case diagrams
- Activity diagrams

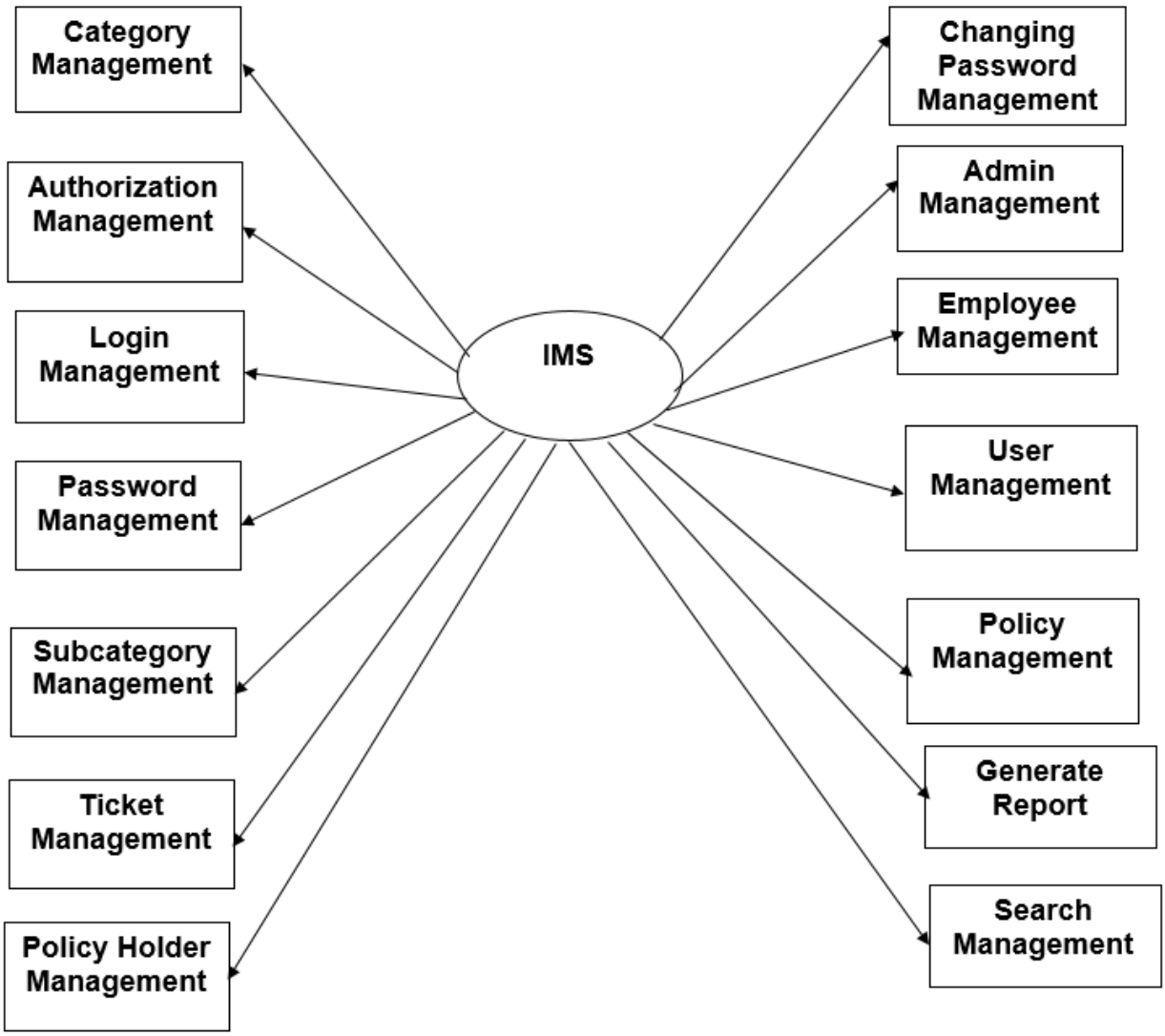
Data flow diagram explains the overall view of utilization of user's data in different levels/modules of the project. Use case diagrams explain main entities and their features. Activity diagrams explain the main activities/processes present in the project**.**

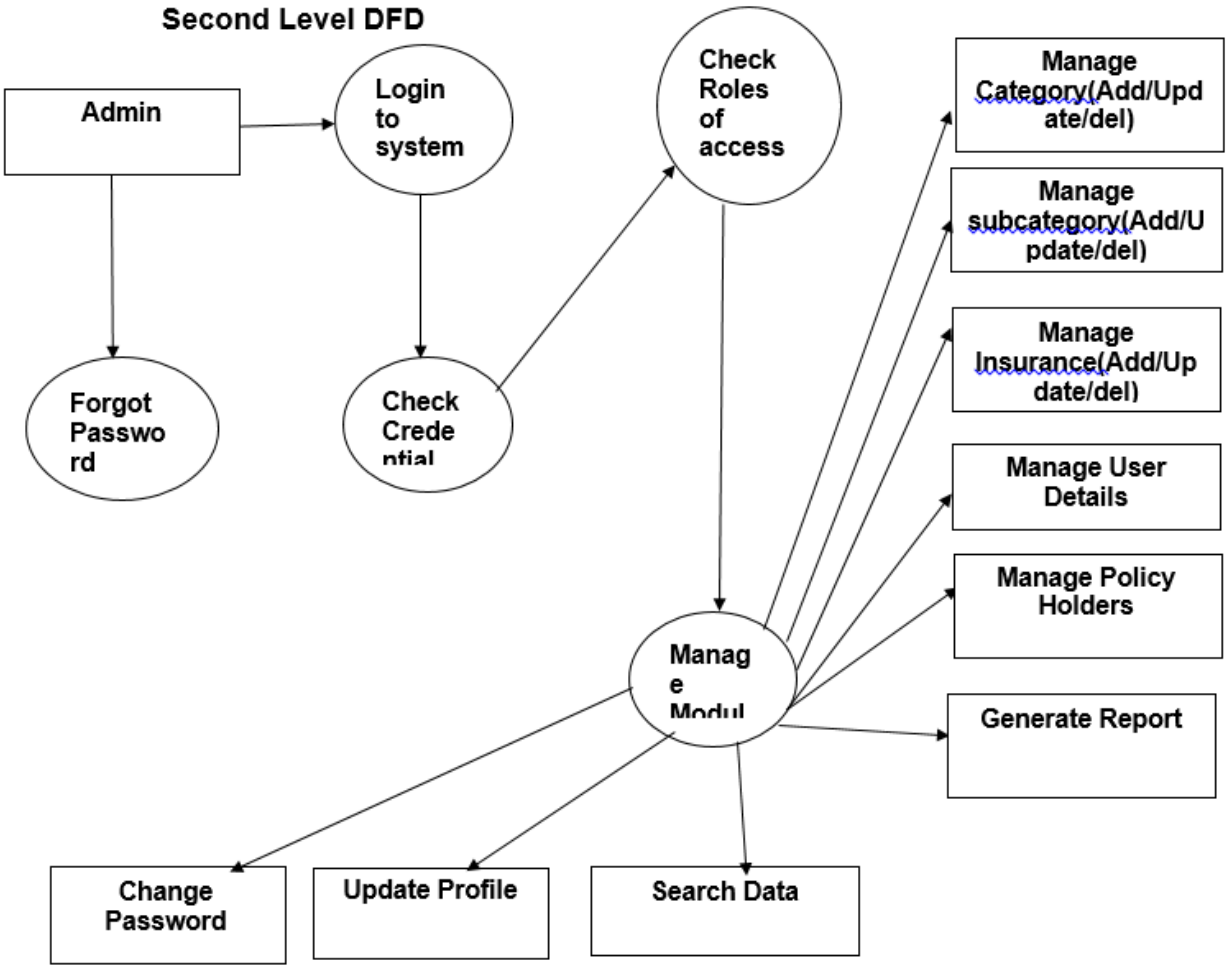
## DATA FLOW DIAGRAM

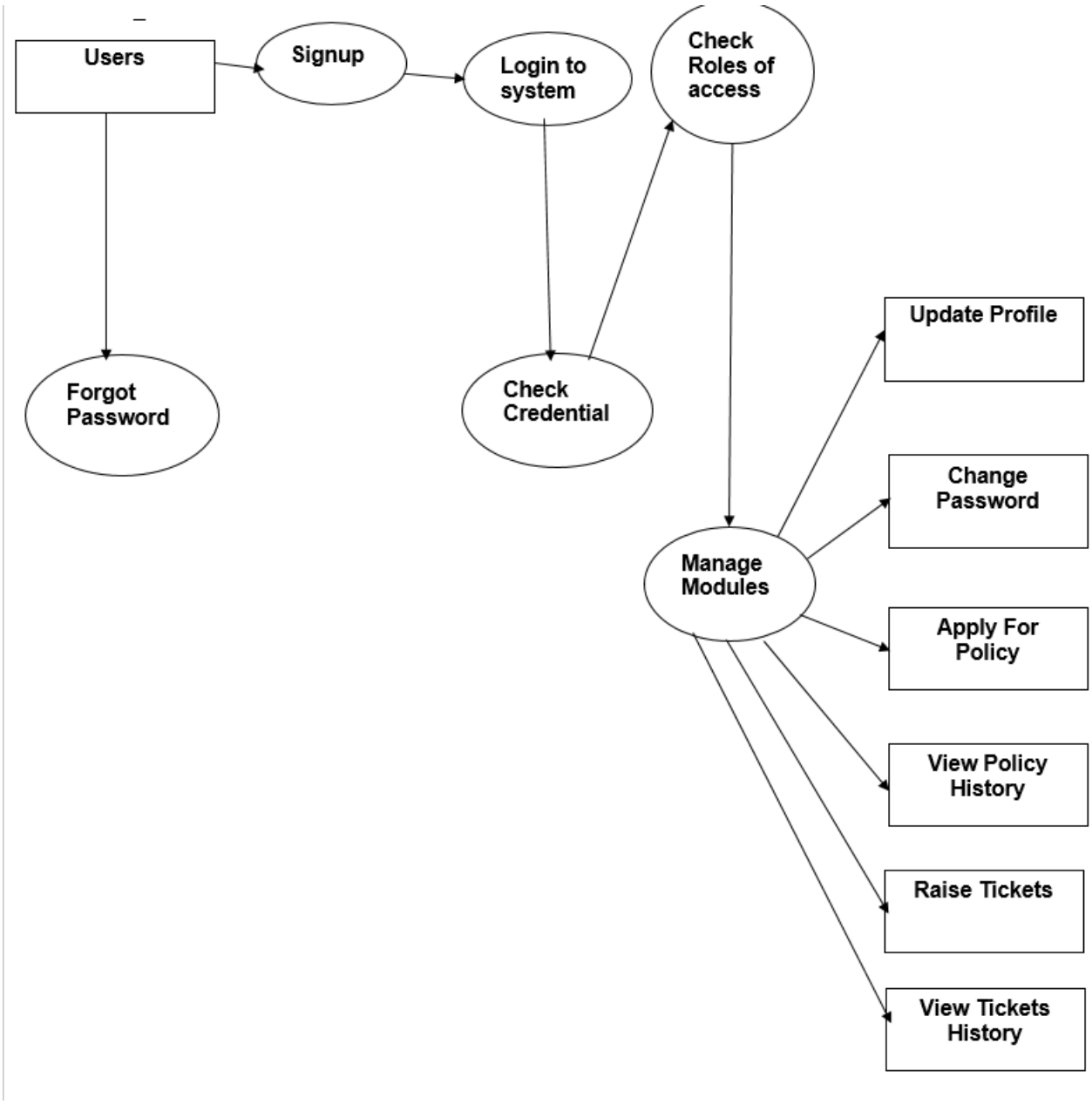
**Zero Level DFD**



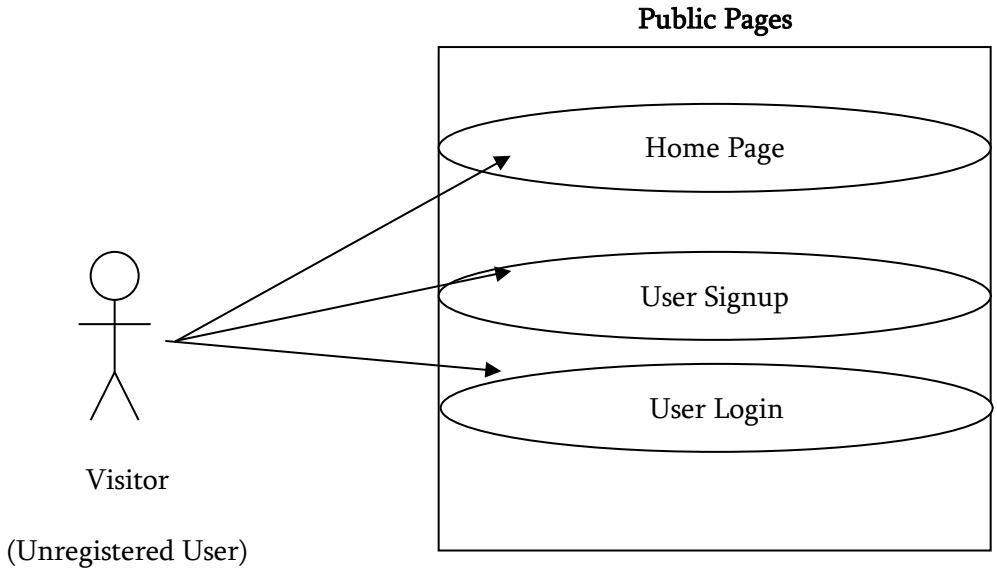
First Level DFD



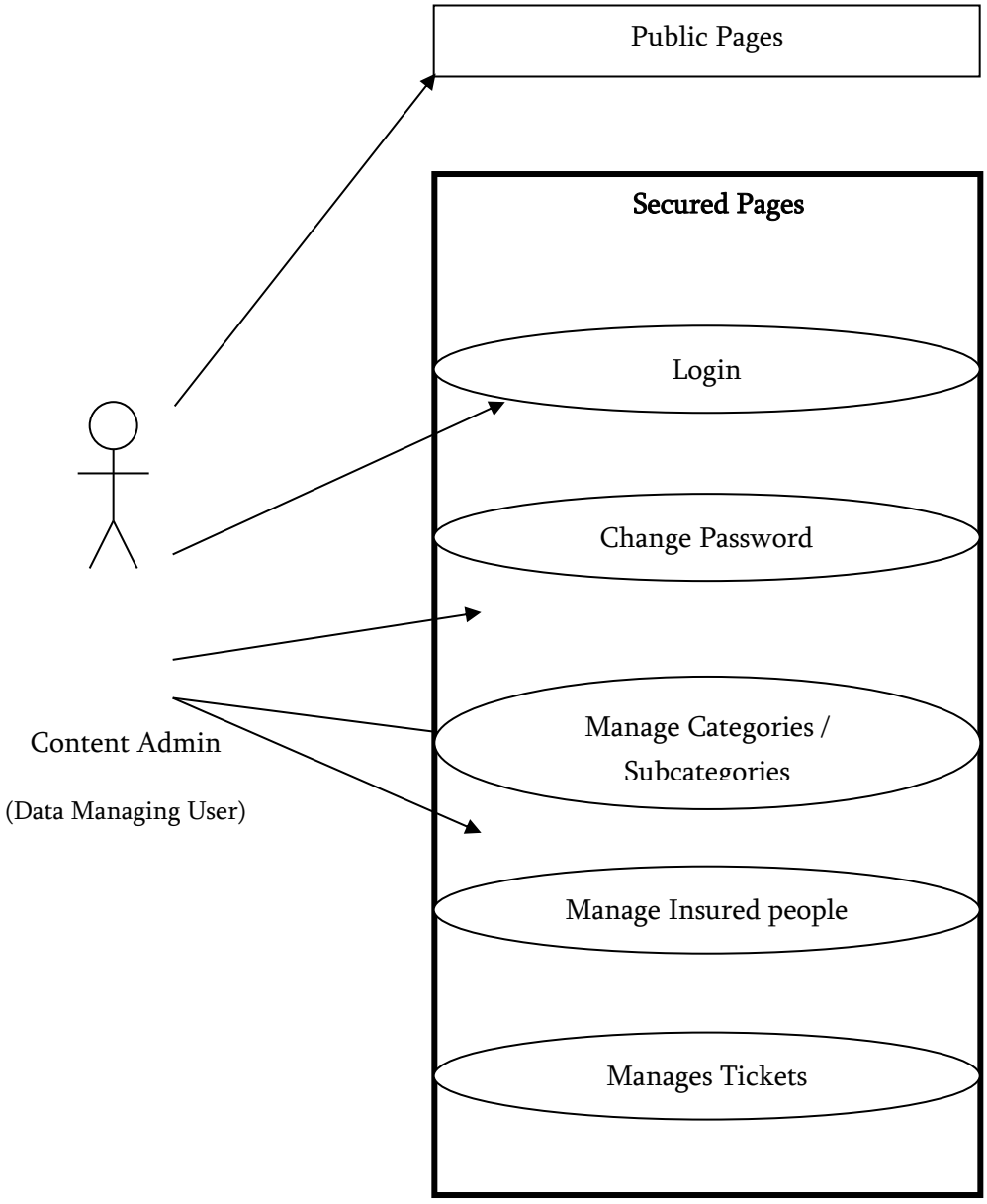

Second Level DFD
Admin
Login to system
Check Roles of access
Manage Category(Add/Update/del)
Manage subcategory(Add/Update/del)
Manage Insurance(Add/Update/del)
Manage User Details
Manage Policy Holders
Generate Report
Forgot Password
Check Credential
Manage Module
Change Password
Update Profile
Search Data


Users
Signup
Login to system
Check Roles of access
Forgot Password
Check Credential
Update Profile
Change Password
Manage Modules
Apply For Policy
View Policy History
Raise Tickets
View Tickets History

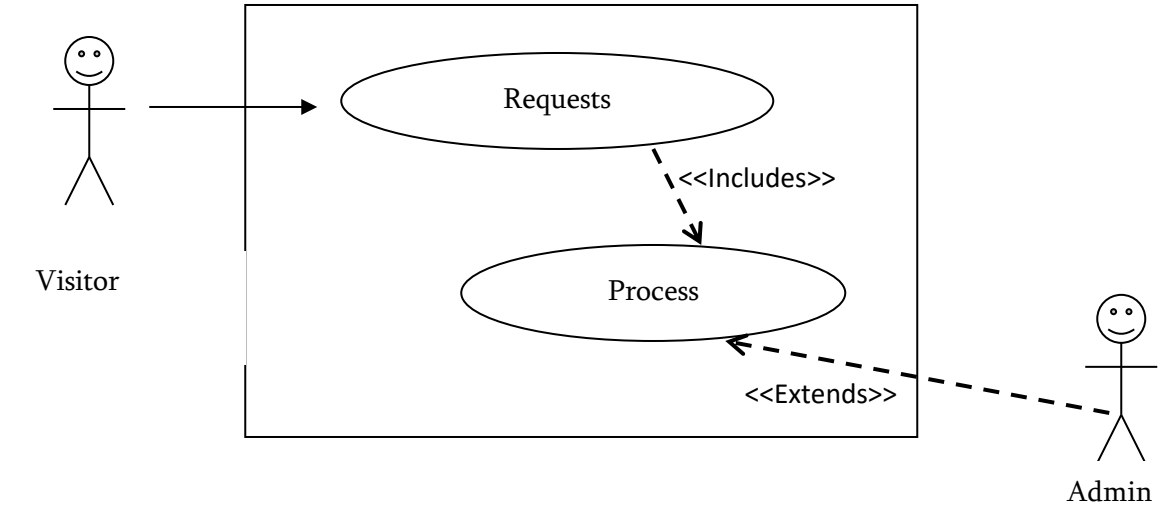
## USE CASE DIAGRAMS Use Case Diagram: Visitor and Facilities

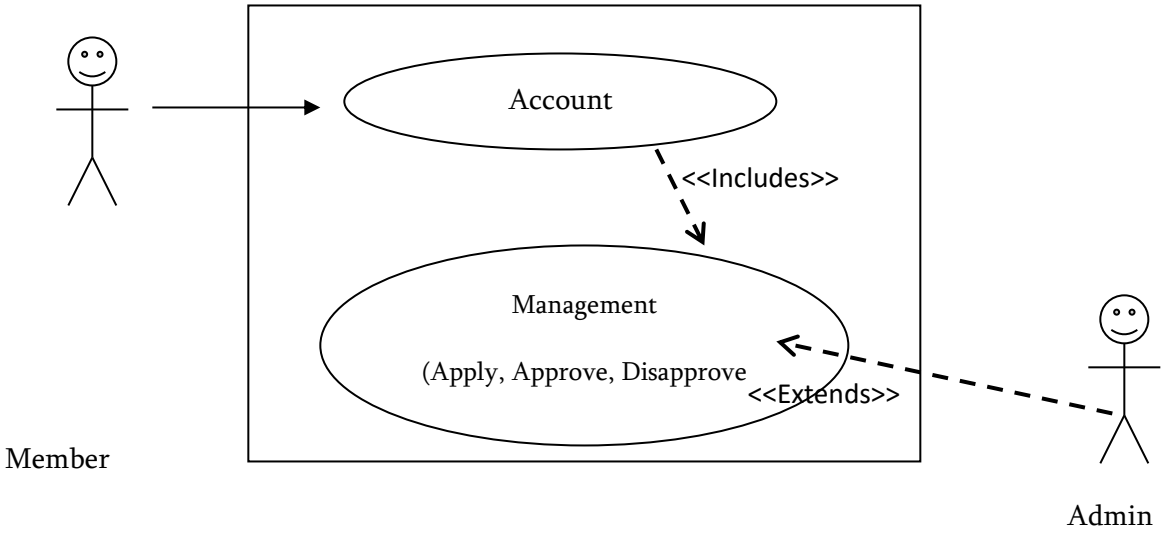


## Use Case Diagram: Online Registered Members and Facilities



## USE CASE RELATIONSHIP DIAGRAMS



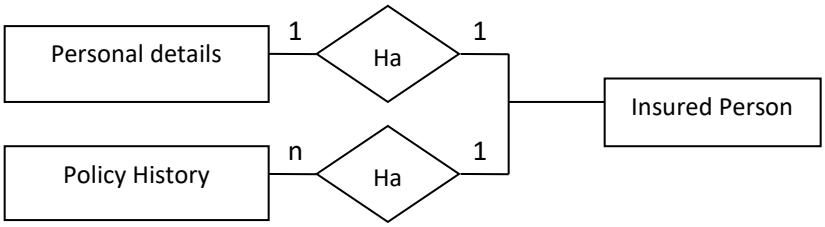
# Entity Relation (ER) Diagrams

## ER (Entity Relation) diagrams

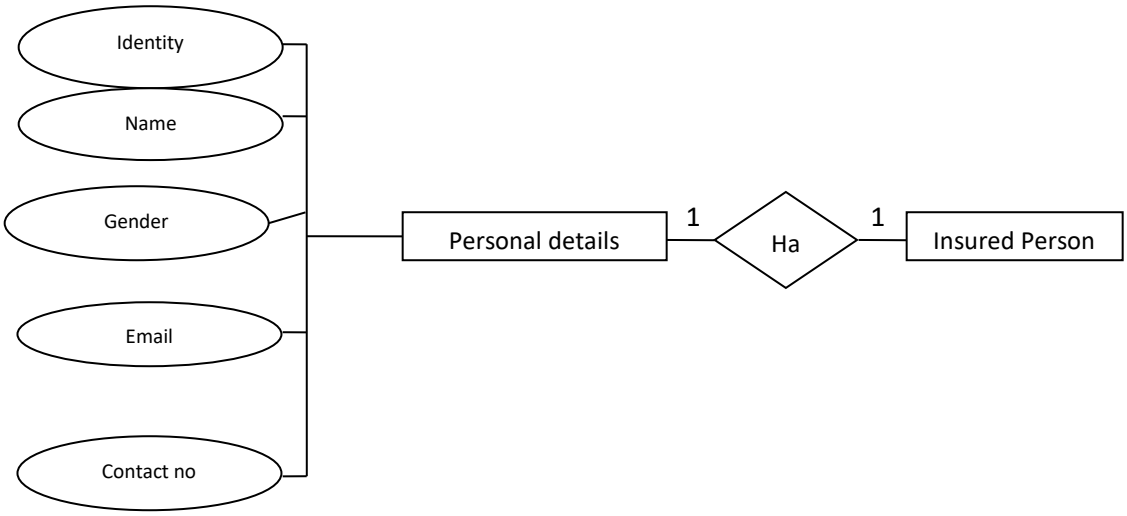
An ER diagram gives a clear idea of logical relation between two or more entities. These diagrams help us to understand the technical design of database involved in the project.

The following relational diagrams show the logical relation between the entities.

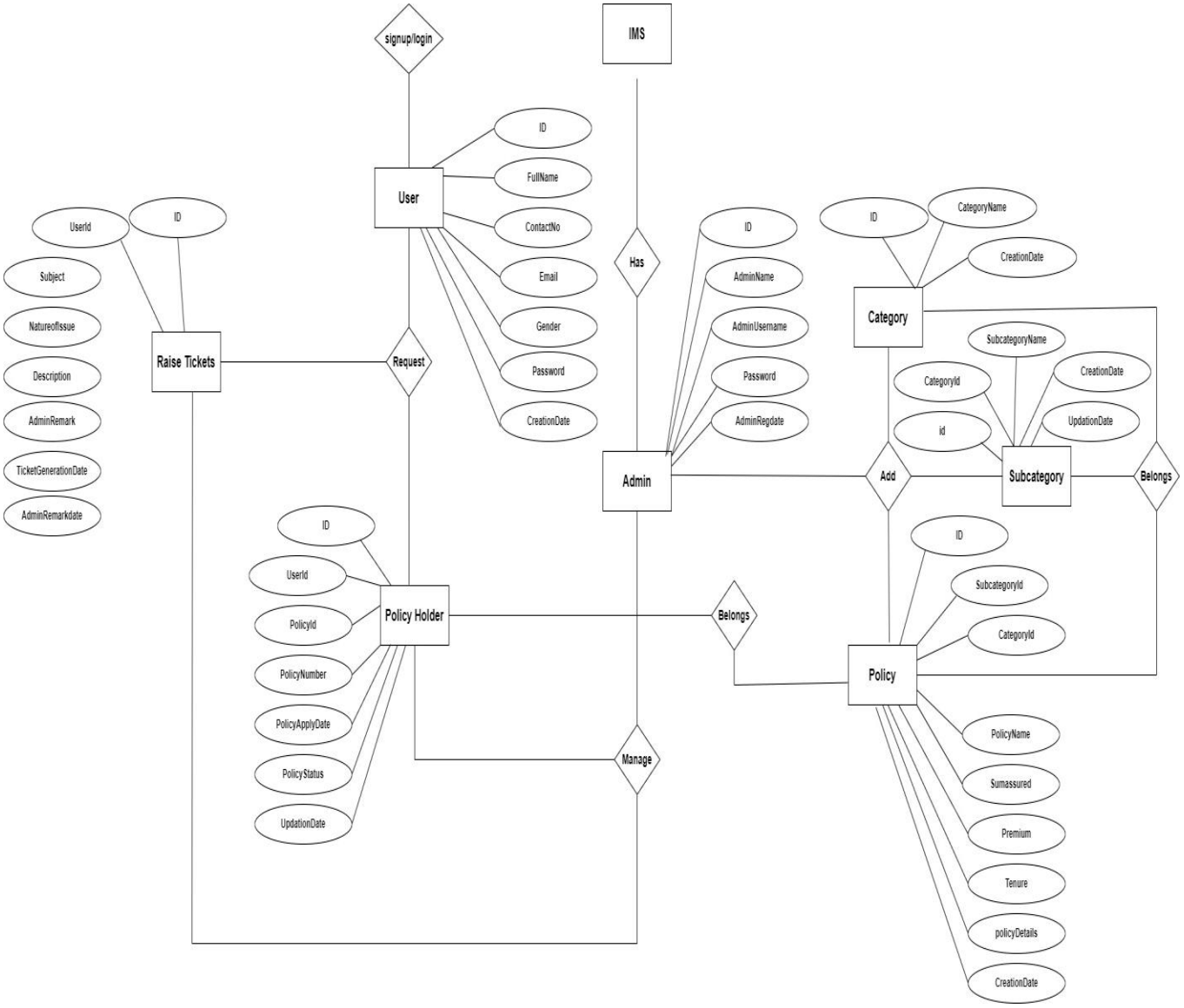
## Insured Person's details diagram



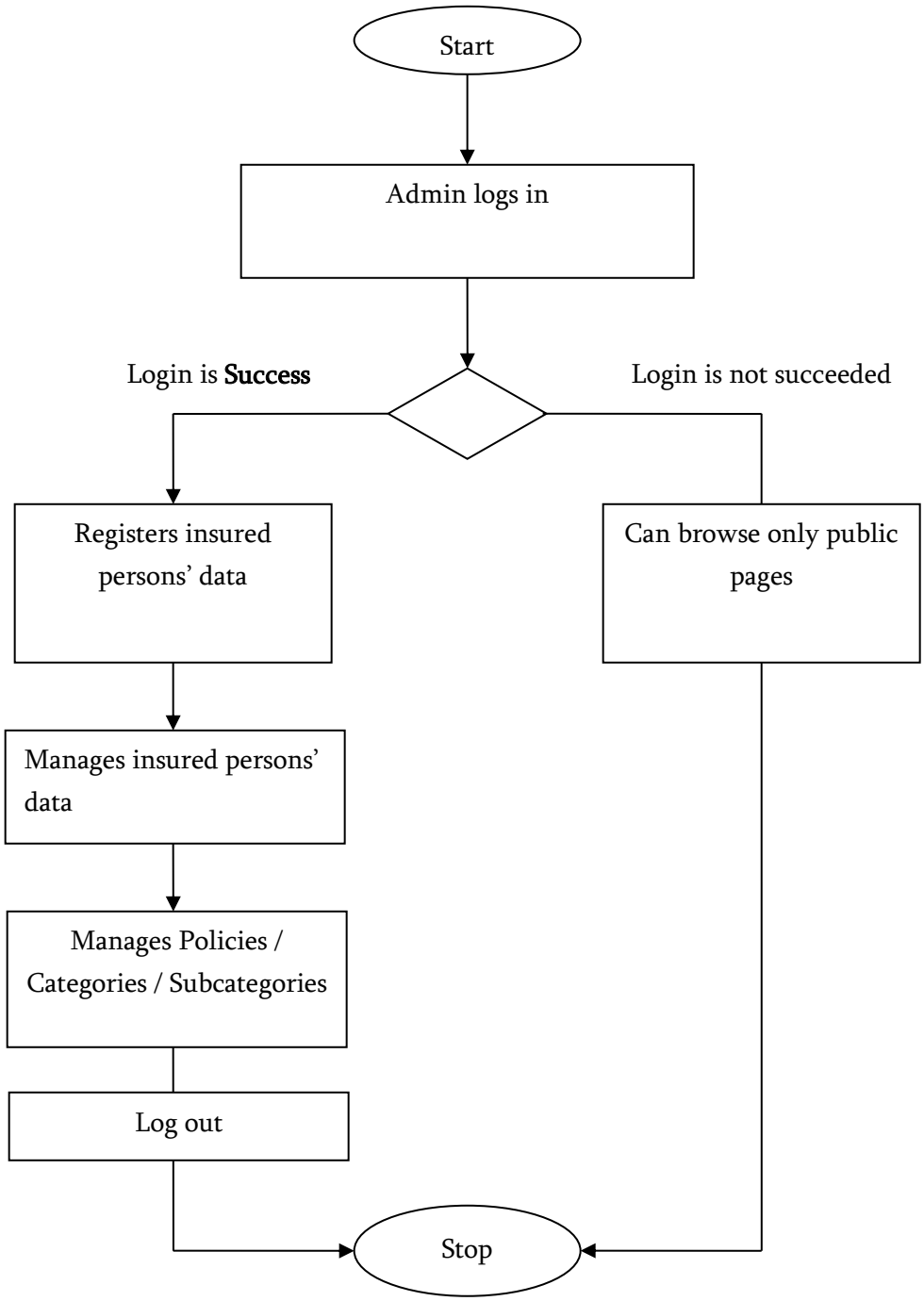

## Insured Person – Personal details relation



## ER- Diagram


signup/login
IMS
User
ID
FullName
ContactNo
Email
Gender
Password
CreationDate
Has
ID
AdminName
AdminUsername
Password
AdminRegdate
Admin
Raise Tickets
UserId
ID
Subject
Natureofissue
Description
AdminRemark
TicketGenerationDate
AdminRemarkdate
Request
Policy Holder
ID
UserId
PolicyId
PolicyNumber
PolicyApplyDate
PolicyStatus
UpdationDate
Manage
Belongs
Category
ID
CategoryName
CreationDate
Add
Subcategory
SubcategoryName
CategoryId
CreationDate
id
UpdationDate
Belongs
Policy
ID
SubcategoryId
CategoryId
PolicyName
Sumassured
Premium
Tenure
policyDetails
CreationDate

## FUNCTIONAL FLOW DIAGRAM



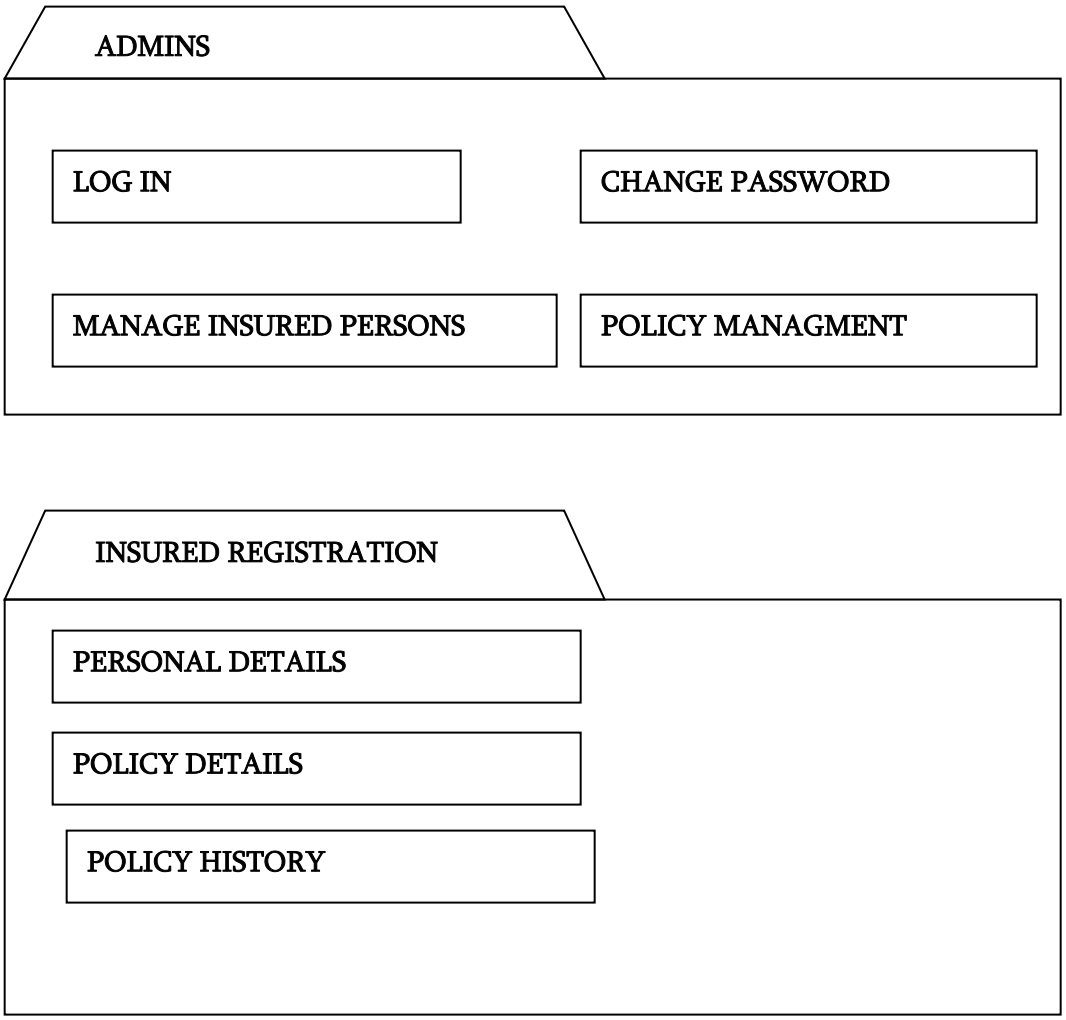

## Subsystem decomposition

This section shows the all-functional objects in the project and their functionality.

The following list of functional object as identified in the project.

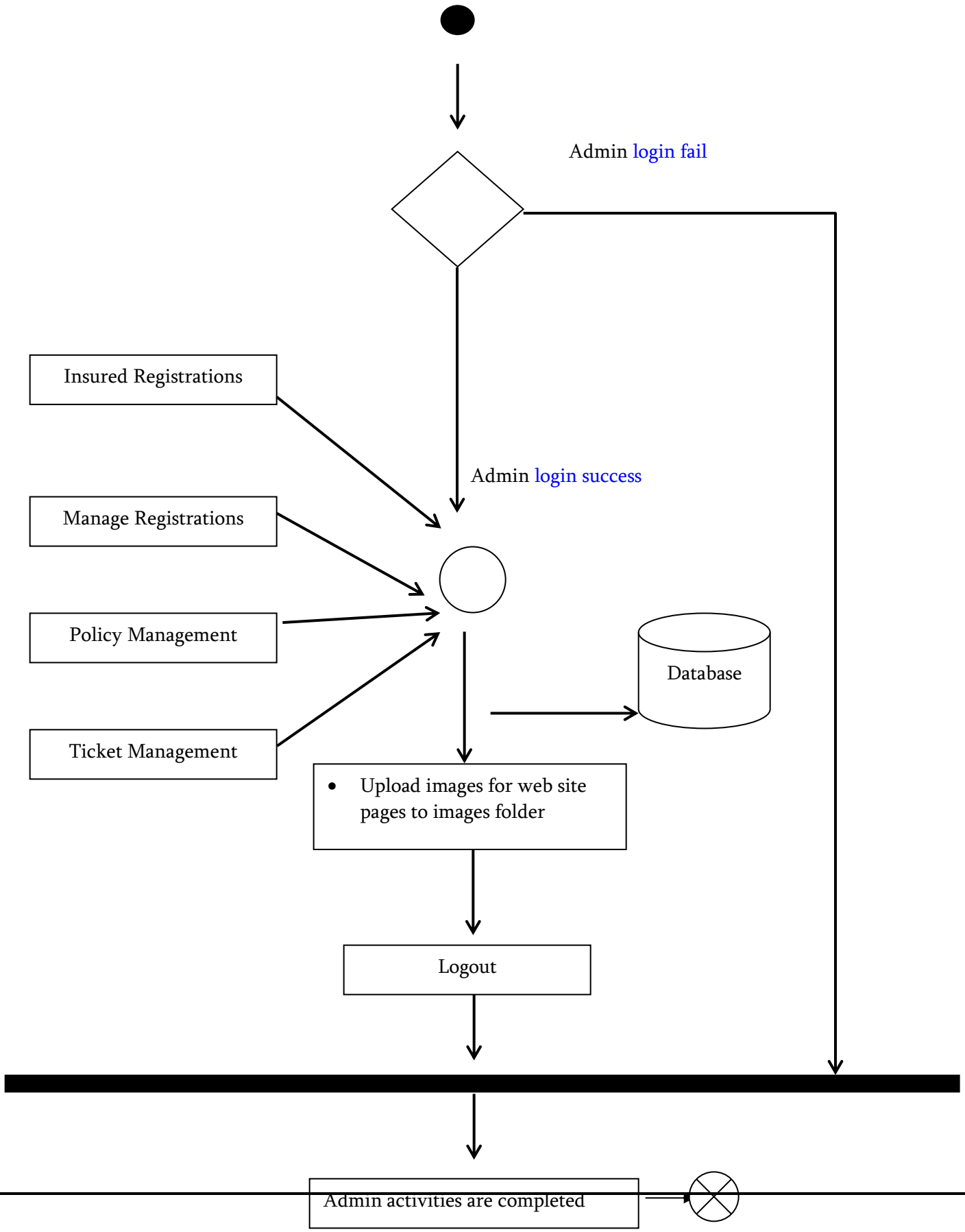
- Login
- Manage Categories / Subcategories / Policies
- Manages Insure Persons data
- Manages Articles

Insured person's



## Activity diagrams

### ACTIVITY DIAGRAM OF ADMIN ACTIVITIES

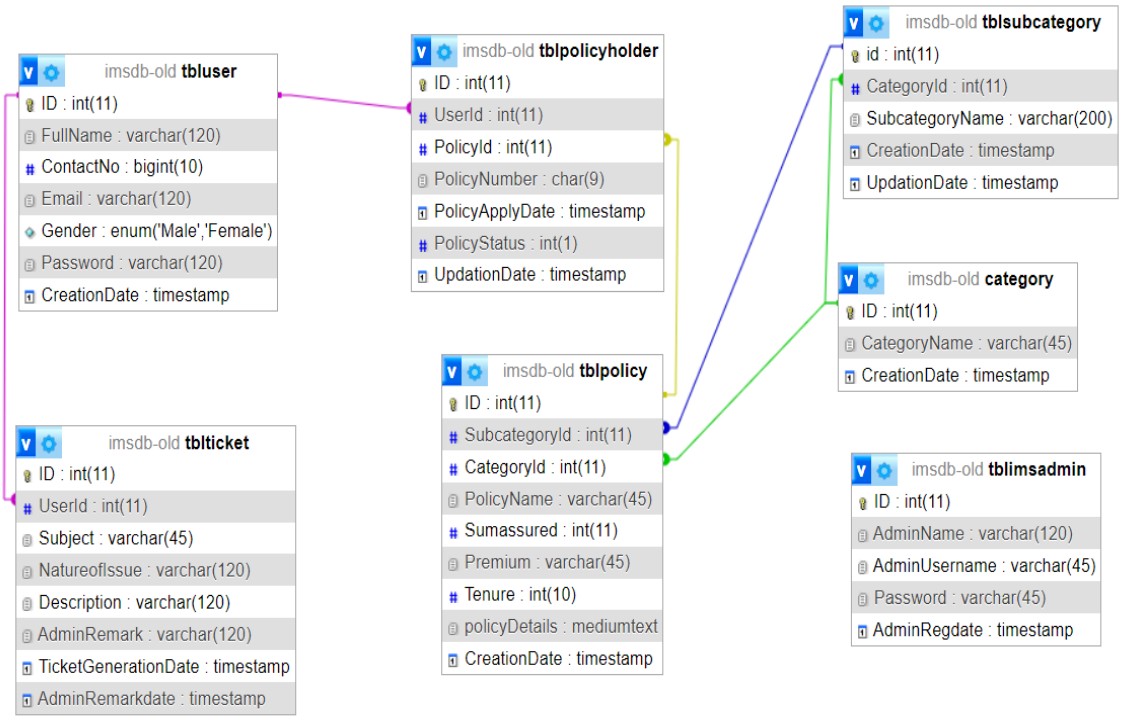


## Database Design

The data in the system has to be stored and retrieved from database. Designing the database is part of system design. Data elements and data structures to be stored have been identified at analysis stage. They are structured and put together to design the data storage and retrieval system.

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make database access easy, quick, inexpensive and flexible for the user. Relationships are established between the data items and unnecessary data items are removed. Normalization is done to get an internal consistency of data and to have minimum redundancy and maximum stability. This ensures minimizing data storage required, minimizing chances of data inconsistencies and optimizing for updates. The MySQL database has been chosen for developing the relevant databases.

## Database Table Relationship




## MySQL Table and their Structure

### Admin Table: tblimsdmin table Structure

This table store login details of admin.

**tblimsadmin**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | **ID** | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | **AdminName** | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 3 | **AdminUsername** | varchar(45) | latin1_swedish_ci | | Yes | *NULL* | | |
| 4 | **Password** | varchar(45) | latin1_swedish_ci | | Yes | *NULL* | | |
| 5 | **AdminRegdate** | timestamp | | | Yes | current_timestamp() | | |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| **PRIMARY** | BTREE | Yes | No | ID | 0 | A | No | |

**Category Table: category table Structure**

This table store the details of category of policy.

**category**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | **ID** | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | **CategoryName** | varchar(45) | latin1_swedish_ci | | Yes | *NULL* | | |
| 3 | **CreationDate** | timestamp | | | Yes | current_timestamp() | | |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| **PRIMARY** | BTREE | Yes | No | ID | 5 | A | No | |

**Subcategory Table: tblsubcategory table Structure**

This table store the details of subcategory of policy.

**tblsubcategory**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | **id** | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | **CategoryId** | char(5) | latin1_swedish_ci | | No | *None* | | |
| 3 | **SubcategoryName** | varchar(200) | latin1_swedish_ci | | No | *None* | | |
| 4 | **CreationDate** | timestamp | | | Yes | current_timestamp() | | |
| 5 | **UpdationDate** | timestamp | | | Yes | *NULL* | | ON UPDATE CURRENT_TIMESTAMP() |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| **PRIMARY** | BTREE | Yes | No | id | 32 | A | No | |

**Policy Table: tblpolicy table Structure**

This table store the detail of policy.

**tblpolicy**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | **ID** | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | **SubcategoryId** | varchar(50) | latin1_swedish_ci | | No | *None* | | |
| 3 | **CategoryId** | varchar(25) | latin1_swedish_ci | | Yes | *NULL* | | |
| 4 | **PolicyName** | varchar(45) | latin1_swedish_ci | | Yes | *NULL* | | |
| 5 | **Sumassured** | int(11) | | | Yes | *NULL* | | |
| 6 | **Premium** | varchar(45) | latin1_swedish_ci | | Yes | *NULL* | | |
| 7 | **Tenure** | int(10) | | | Yes | *NULL* | | |
| 8 | **policyDetails** | mediumtext | latin1_swedish_ci | | Yes | *NULL* | | |
| 9 | **CreationDate** | timestamp | | | No | current_timestamp() | | ON UPDATE CURRENT_TIMESTAMP() |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| **PRIMARY** | BTREE | Yes | No | ID | 2 | A | No | |

**User Table: tbluser table Structure**

This table store the details of user which is register.

**tbluser**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | **ID** | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | **FullName** | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 3 | **ContactNo** | bigint(10) | | | Yes | *NULL* | | |
| 4 | **Email** | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 5 | **Gender** | enum('Male', 'Female') | latin1_swedish_ci | | Yes | *NULL* | | |
| 6 | **Password** | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 7 | **CreationDate** | timestamp | | | No | current_timestamp() | | |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| **PRIMARY** | BTREE | Yes | No | ID | 1 | A | No | |

**Policy Holder Table: tblpolicyholder table Structure**
This table store the details of policy holder.

**tblpolicyholder**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | ID | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | UserId | char(20) | latin1_swedish_ci | | Yes | *NULL* | | |
| 3 | PolicyId | char(20) | latin1_swedish_ci | | Yes | *NULL* | | |
| 4 | PolicyNumber | char(9) | latin1_swedish_ci | | Yes | *NULL* | | |
| 5 | PolicyApplyDate | timestamp | | | Yes | current_timestamp() | | |
| 6 | PolicyStatus | int(1) | | | Yes | *NULL* | | |
| 7 | UpdationDate | timestamp | | | Yes | *NULL* | | ON UPDATE CURRENT_TIMESTAMP() |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| PRIMARY | BTREE | Yes | No | ID | 3 | A | No | |

**Ticket Table: tblticket table Structure**
This table store the details of tickets which is raised by users.

**tblticket**

| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra |
|---|---|---|---|---|---|---|---|---|
| 1 | ID | int(11) | | | No | *None* | | AUTO_INCREMENT |
| 2 | UserId | varchar(25) | latin1_swedish_ci | | Yes | *NULL* | | |
| 3 | Subject | varchar(45) | latin1_swedish_ci | | Yes | *NULL* | | |
| 4 | NatureofIssue | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 5 | Description | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 6 | AdminRemark | varchar(120) | latin1_swedish_ci | | Yes | *NULL* | | |
| 7 | TicketGenerationDate | timestamp | | | Yes | current_timestamp() | | |
| 8 | AdminRemarkdate | timestamp | | | Yes | *NULL* | | ON UPDATE CURRENT_TIMESTAMP() |

Indexes

| Keyname | Type | Unique | Packed | Column | Cardinality | Collation | Null | Comment |
|---|---|---|---|---|---|---|---|---|
| PRIMARY | BTREE | Yes | No | ID | 3 | A | No | |

## Output Design

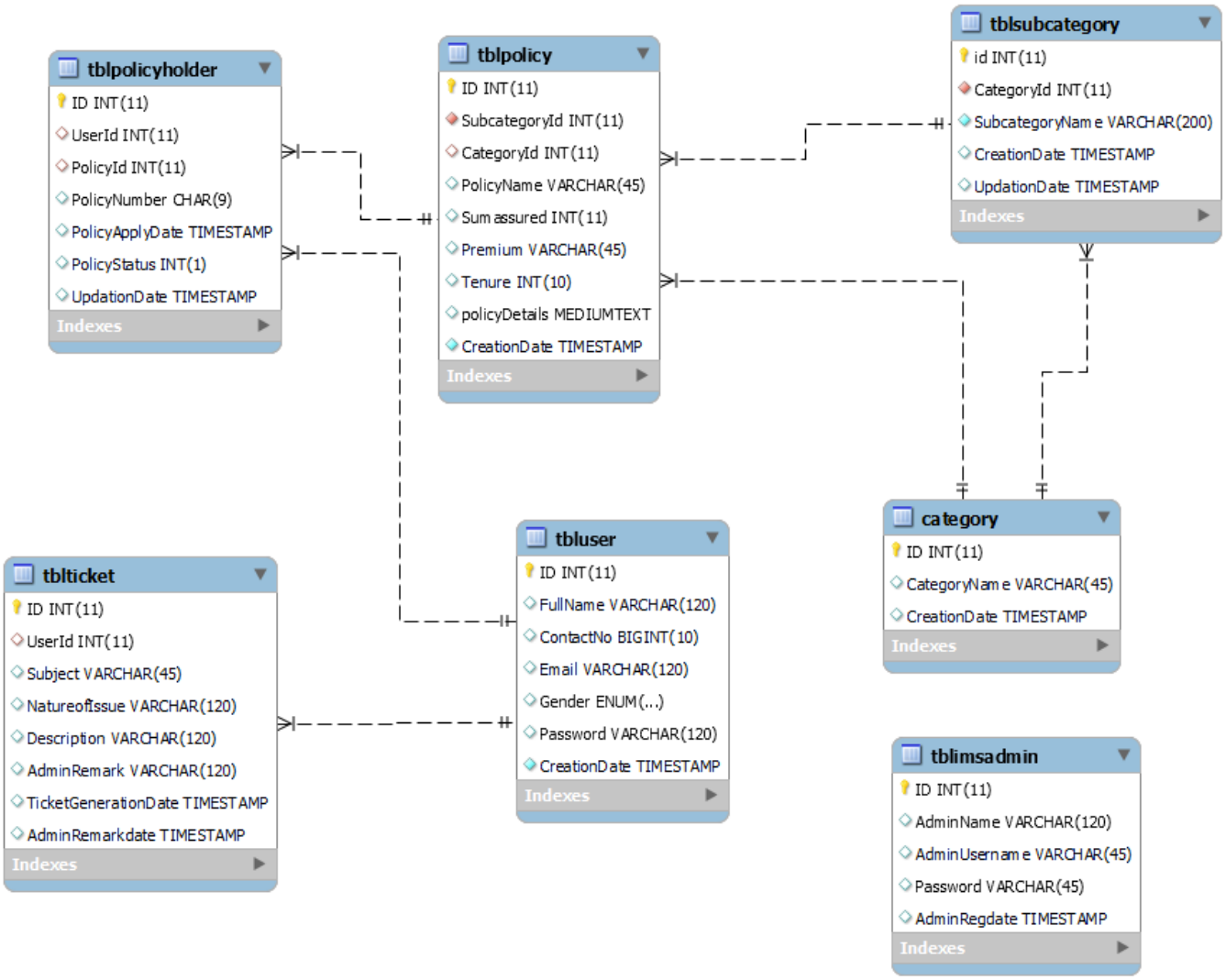
Designing computer output should proceed in an organized, well throughout manner; the right output element is designed so that people will find the system whether or executed. When we design an output we must identify the specific output that is needed to meet the system. The usefulness of the new system is evaluated on the basis of their output.

Once the output requirements are determined, the system designer can decide what to include in the system and how to structure it so that the require output can be produced. For the proposed software, it is necessary that the output reports be compatible in format with the existing reports. The output must be concerned to the overall performance and the system's working, as it should. It consists of developing specifications and procedures for data preparation, those steps necessary to put the inputs and the desired output, ie maximum user friendly. Proper messages and appropriate directions can control errors committed by users.

The output design is the key to the success of any system. Output is the key between the user and the sensor. The output must be concerned to the system's working, as it should.

Output design consists of displaying specifications and procedures as data presentation. User never left with the confusion as to what is happening without appropriate error and acknowledges message being received. Even an unknown person can operate the system without knowing anything about the system.

## Class Diagram



# SYSTEM ANALYSIS

## Feasibility study

- **Functional Feasibility**

The proposed/developed system is a web application and does not require any additional software requirements except hosting. A web hosting company hosts the web application in their web servers. Any client (end-user) with an internet connection and a web browsing software (at client side) can immediately start accessing/using the system. So, the system does not need any installation/setup procedure. Hence, the system is functionally feasible.

- **Economic Feasibility**

Author (project stakeholder) does not need to purchase any software and hardware to host the developed system. Web hosting companies purchase and maintain all necessary hardware and software for hosting the websites. Author has to pay only hosting charges to the hosting company. When it comes to the end-user, he/she does not need to purchase any software to use the website. Now-a-days every computer's operating system is providing built in web browser and so many web browsing software products are available in the market for free.

Only the cost that the author should bare is cost of development. Before starting the development, the development team estimates cost of development depends on the features asked by the client. If any new features to be added as per the new requirements of the client, cost of new additions are submitted to the client and after getting the confirmation from the client the new features will be added by the development team. Development team will take care of in-time delivery of the project to avoid excess cost of development due to delay.

Hence, the proposed system is economically feasible.

- **Technical feasibility**

This is about acceptance of the new system by the existing end-users and employees of the system and author. The website is aiming at employees and now-a-days all of them are aware of web browsing. If any person does not have any idea of web browsing, he/she will be trained to use the system with in one hour time. So, the

system can easily be accepted by any kind of end-user. Hence the proposed system is technically feasible.

# SYSTEM TESTING

System testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is the process of executing the program with the intent of finding errors and missing operations and also a complete verification to determine whether the objectives are met and the user requirements are satisfied. The ultimate aim is quality assurance.

Tests are carried out and the results are compared with the expected document. In the case of erroneous results, debugging is done. Using detailed testing strategies a test plan is carried out on each module. The various tests performed in "**Network Backup System**" are unit testing, integration testing and user acceptance testing.

**Unit Testing**

The software units in a system are modules and routines that are assembled and integrated to perform a specific function. Unit testing focuses first on modules, independently of one another, to locate errors. This enables, to detect errors in coding and logic that are contained within each module. This testing includes entering data and ascertaining if the value matches to the type and size supported by java. The various controls are tested to ensure that each performs its action as required.

**Integration Testing**

Data can be lost across any interface, one module can have an adverse effect on another, sub functions when combined, may not produce the desired major functions. Integration testing is a systematic testing to discover errors associated within the interface. The objective is to take unit tested modules and build a program structure. All the modules are combined and tested as a whole. Here the Server module and

Client module options are integrated and tested. This testing provides the assurance that the application is well integrated functional unit with smooth transition of data.

**User Acceptance Testing**

User acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the system users at time of developing and making changes whenever required.

# IMPLEMENTATION

Implementation is the stage in the project where the theoretical design is turned into a working system and is giving confidence on the new system for the users that it will work efficiently and effectively. It involves careful planning, investigation of the current system and its constraints on implementation, design of methods to achieve the change over, an evaluation of change over methods. Apart from planning major task of preparing the implementation are education and training of users. The implementation process begins with preparing a plan for the implementation of the system. According to this plan, the activities are to be carried out, discussions made regarding the equipment and resources and the additional equipment has to be acquired to implement the new system. In network backup system no additional resources are needed.

Implementation is the final and the most important phase. The most critical stage in achieving a successful new system is giving the users confidence that the new system will work and be effective. The system can be implemented only after thorough testing is done and if it is found to be working according to the specification. This method also offers the greatest security since the old system can take over if the errors are found or inability to handle certain type of transactions while using the new system.

**User Training**

After the system is implemented successfully, training of the user is one of the most important subtasks of the developer. For this purpose user manuals are prepared and

handled over to the user to operate the developed system. Thus the users are trained to operate the developed system. Both the hardware and software securities are made to run the developed systems successfully in future. In order to put new application system into use, the following activities were taken care of:

- Preparation of user and system documentation
- Conducting user training with demo and hands on
- Test run for some period to ensure smooth switching over the system

The users are trained to use the newly developed functions. User manuals describing the procedures for using the functions listed on menu are circulated to all the users. It is confirmed that the system is implemented up to users need and expectations.

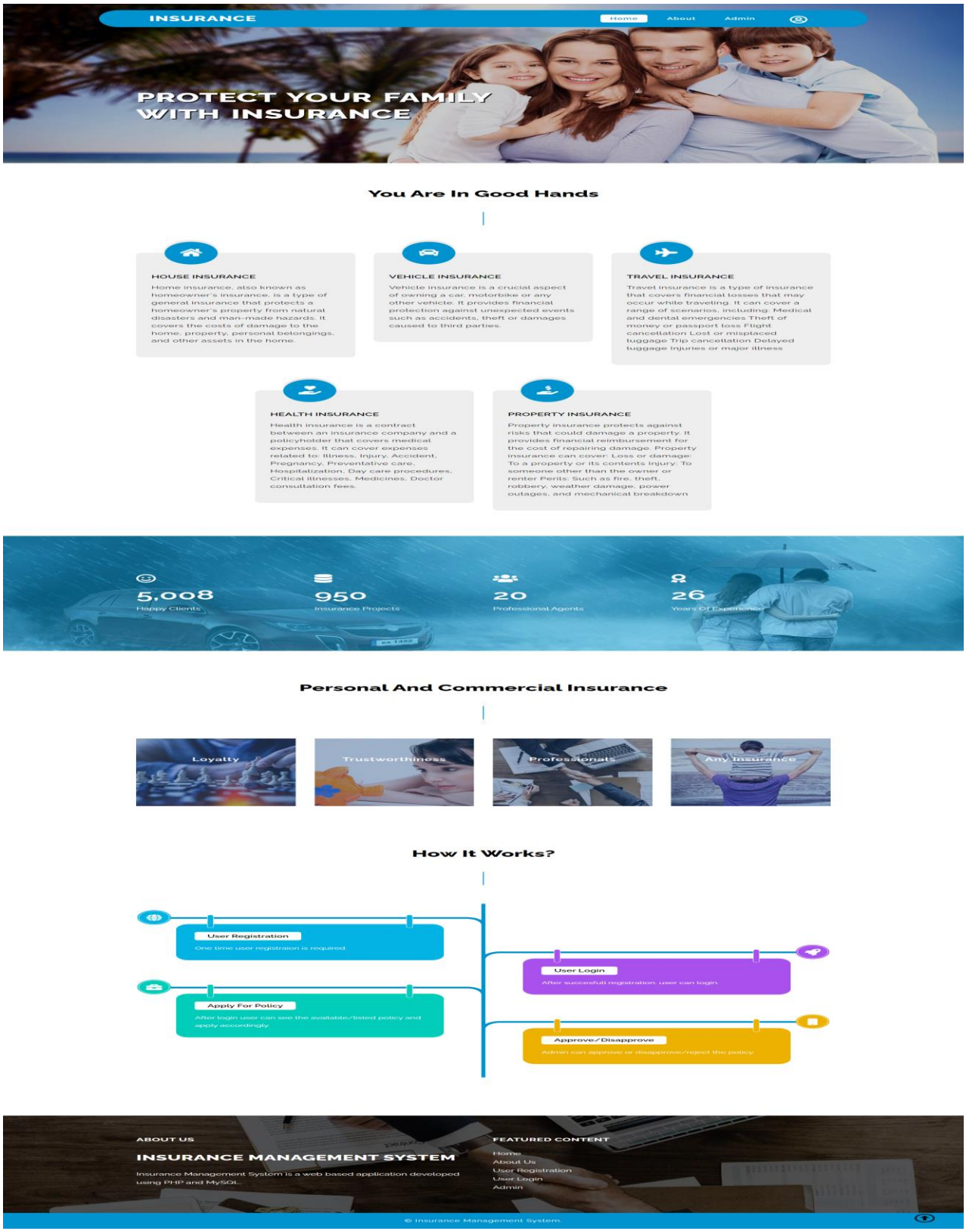
**Security and Maintenance**

Maintenance involves the software industry captive, typing up system resources .It means restoring something to its original condition. Maintenance follows conversion to the extend that changes are necessary to maintain satisfactory operations relative to changes in the user's environment. Maintenance often includes minor enhancements or corrections to problems that surface in the system's operation. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the software.

Any system developed should be secured and protected against possible hazards. Security measures are provided to prevent unauthorized access of the database at various levels. An uninterrupted power supply should be so that the power failure or voltage fluctuations will not erase the data in the file

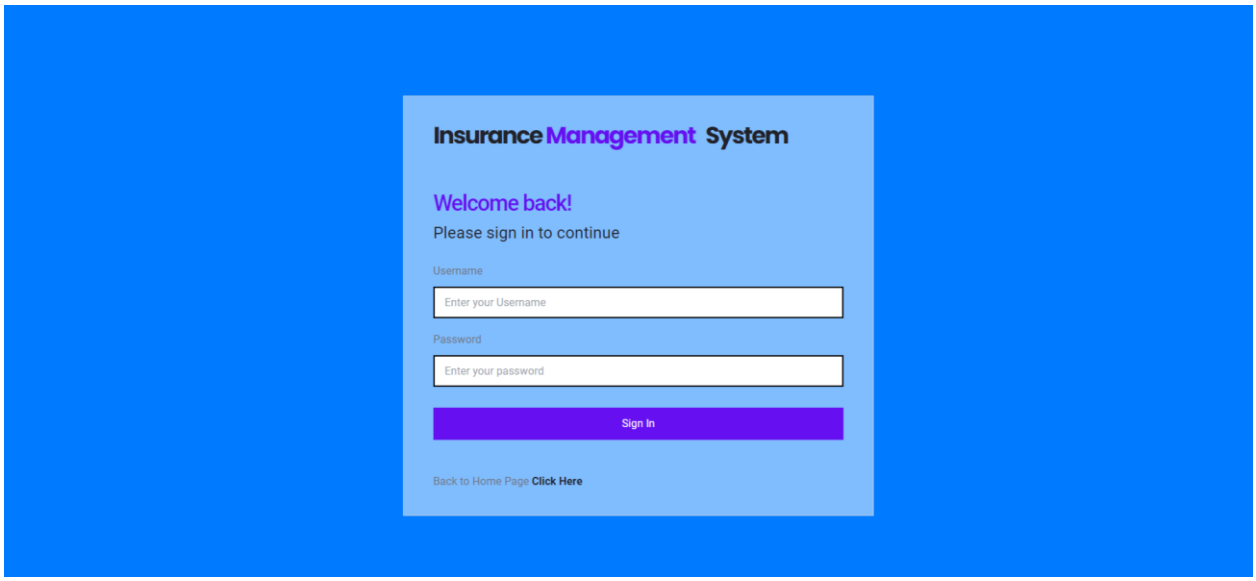
# OUTPUT SCREEN OF PROJECT

## Home Page

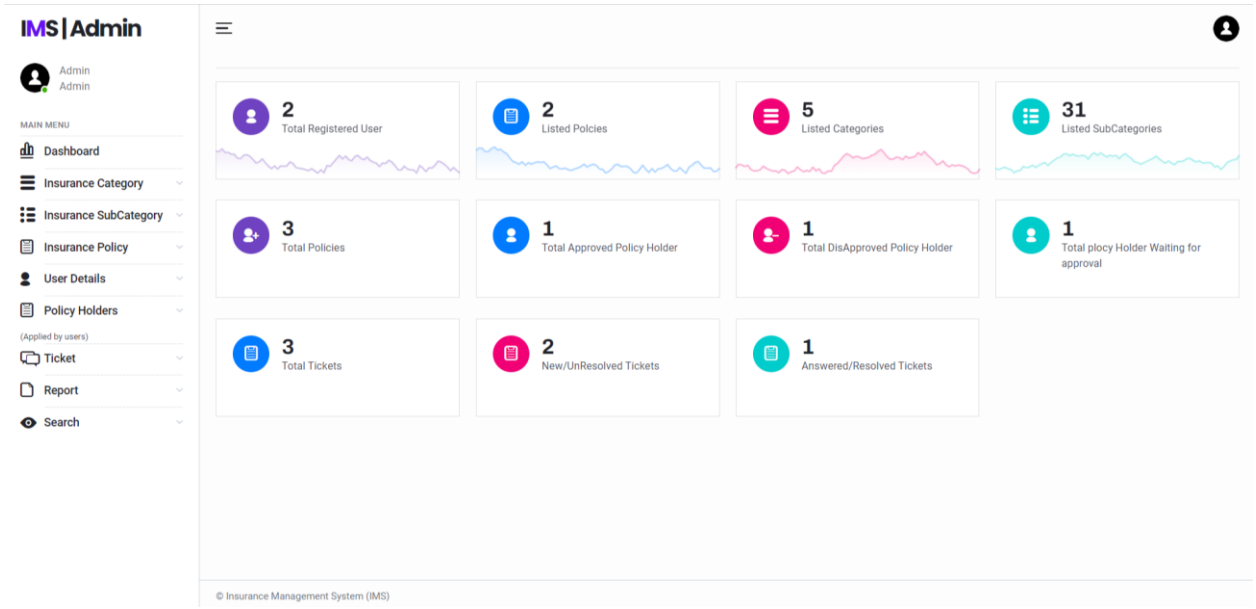


# Admin Panel

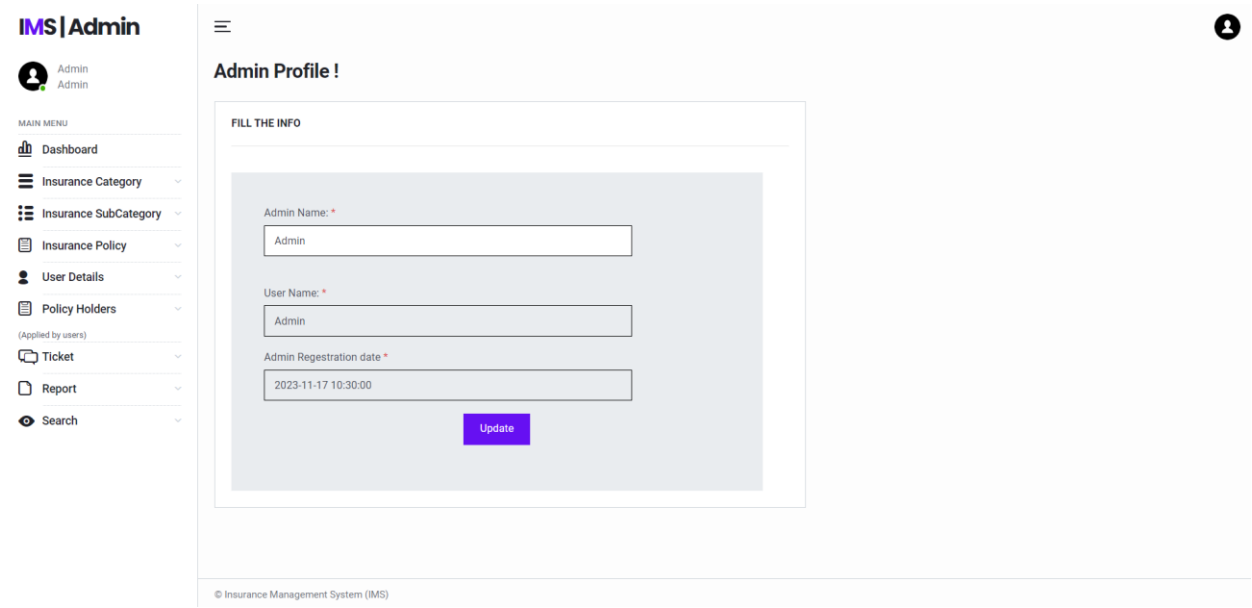
## Login Page



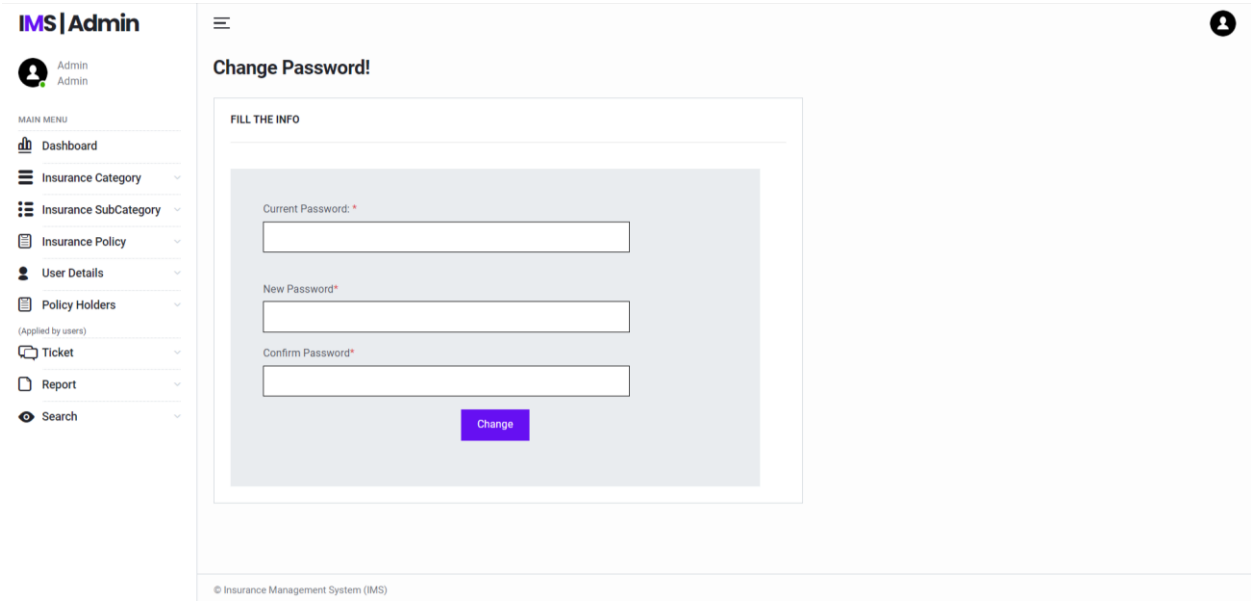

## Dashboard



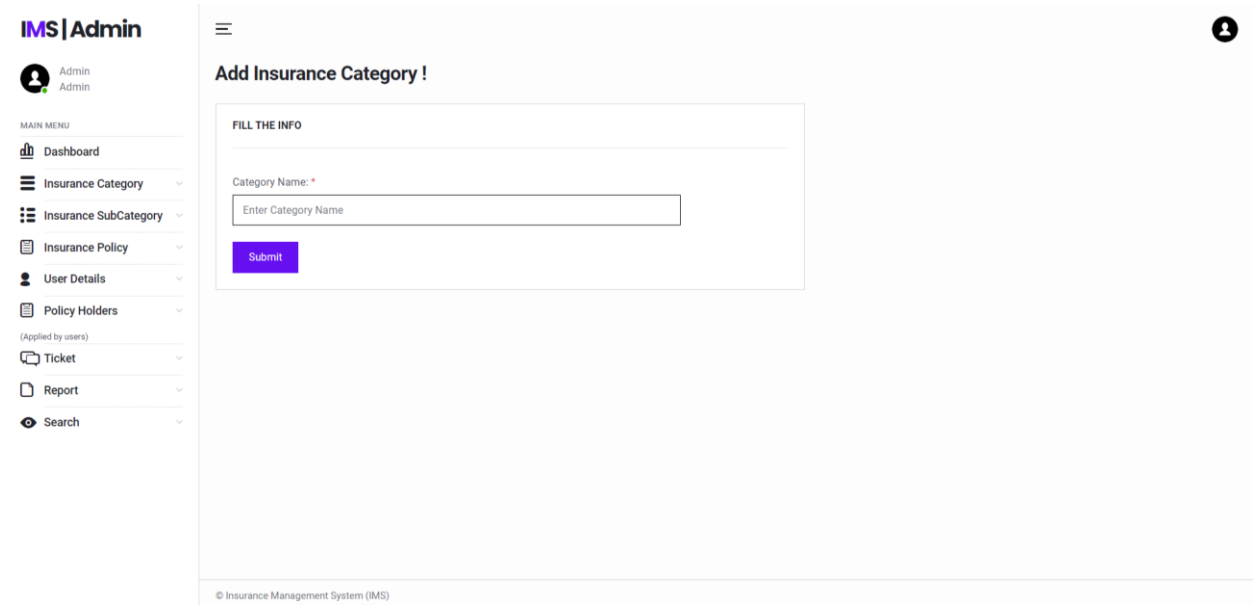
## Profile



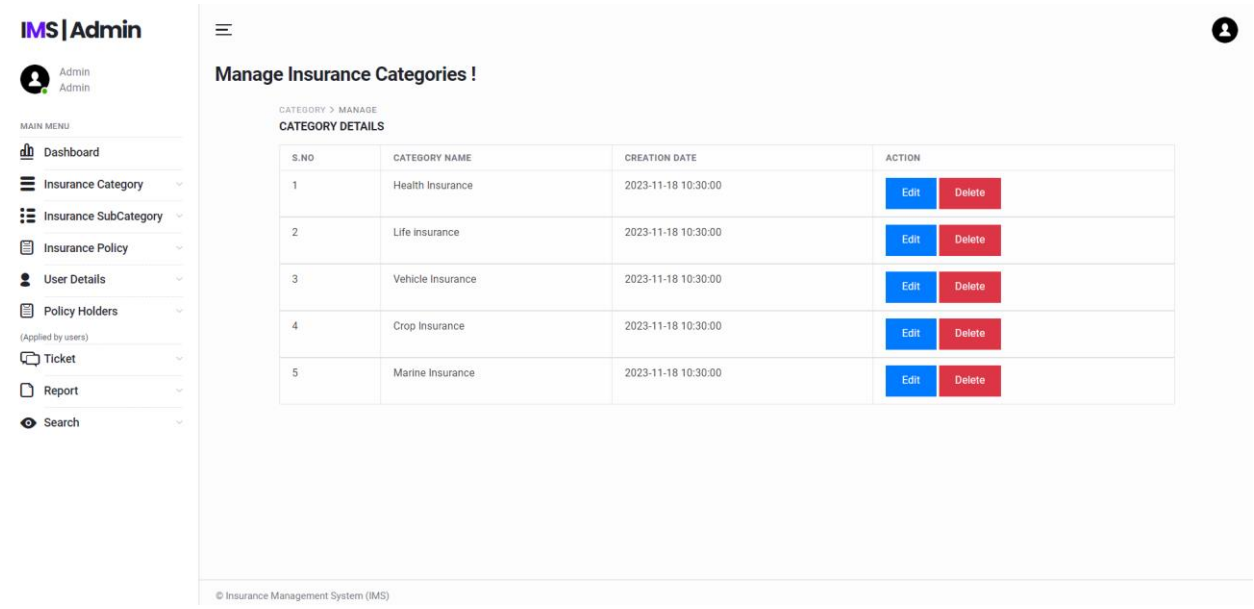

## Change Password



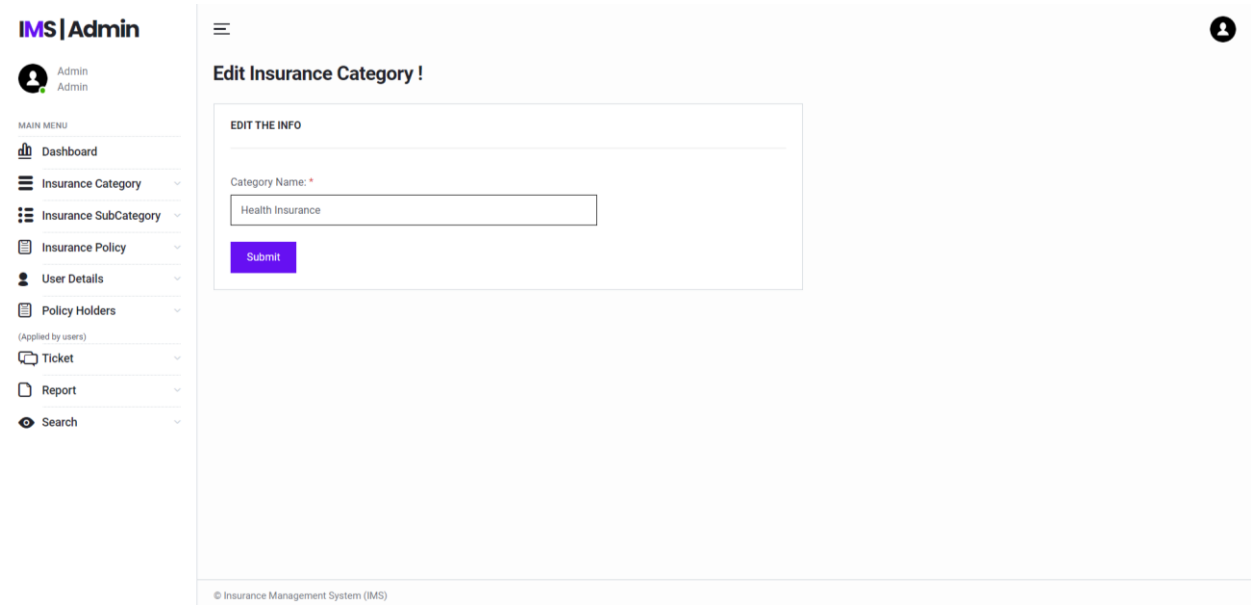
## Add Insurance Category



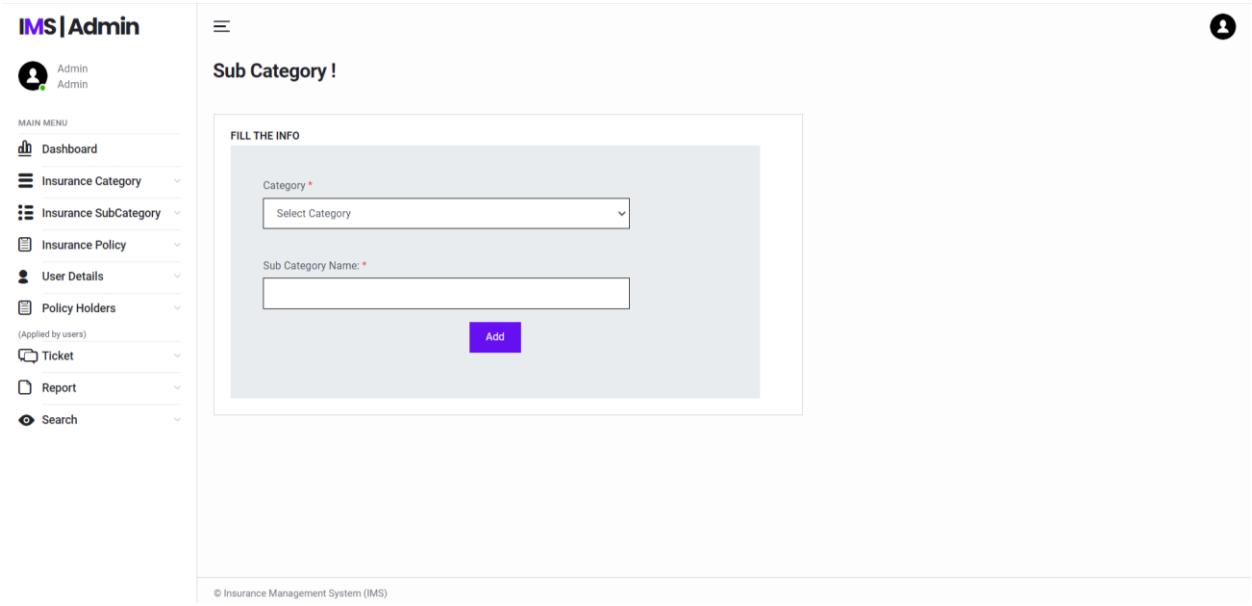

## Manage Category



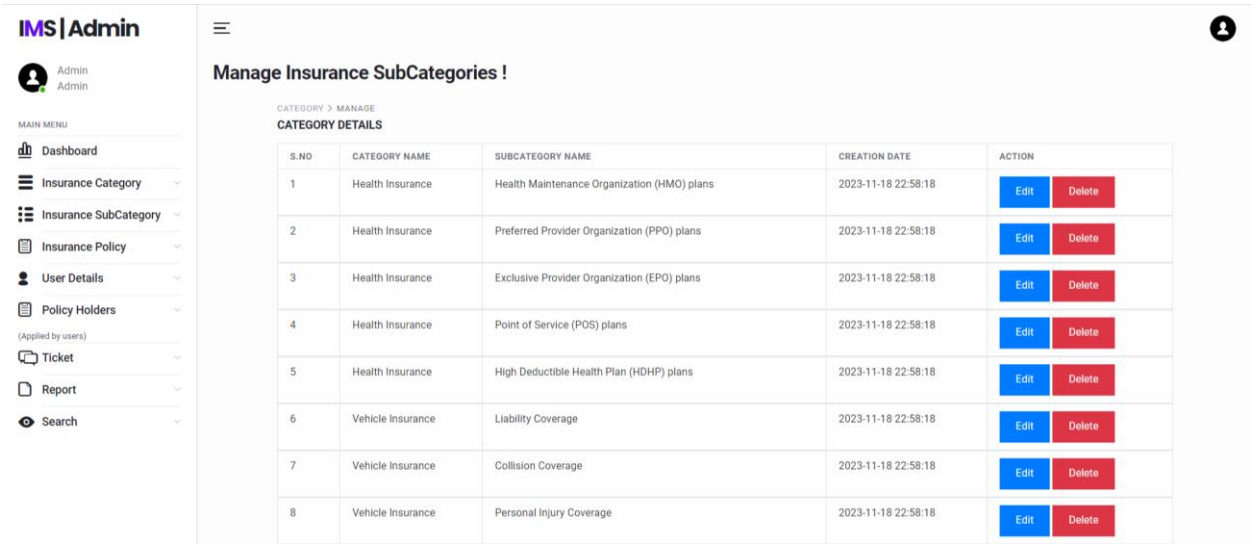
## Update Category



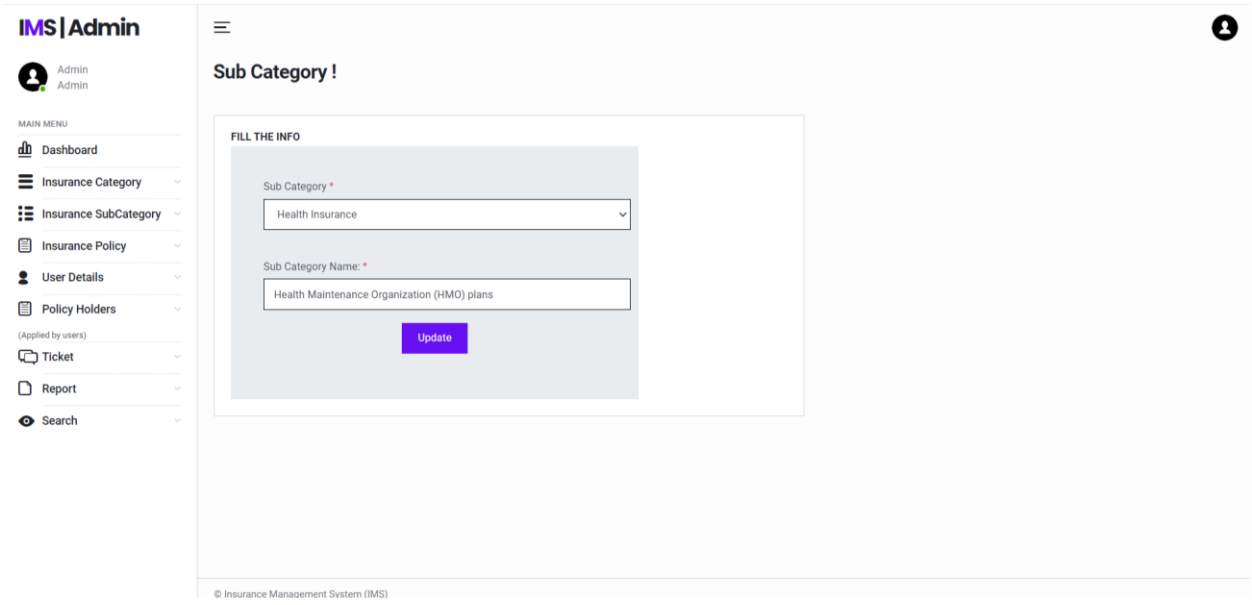

## Insurance Sub Category



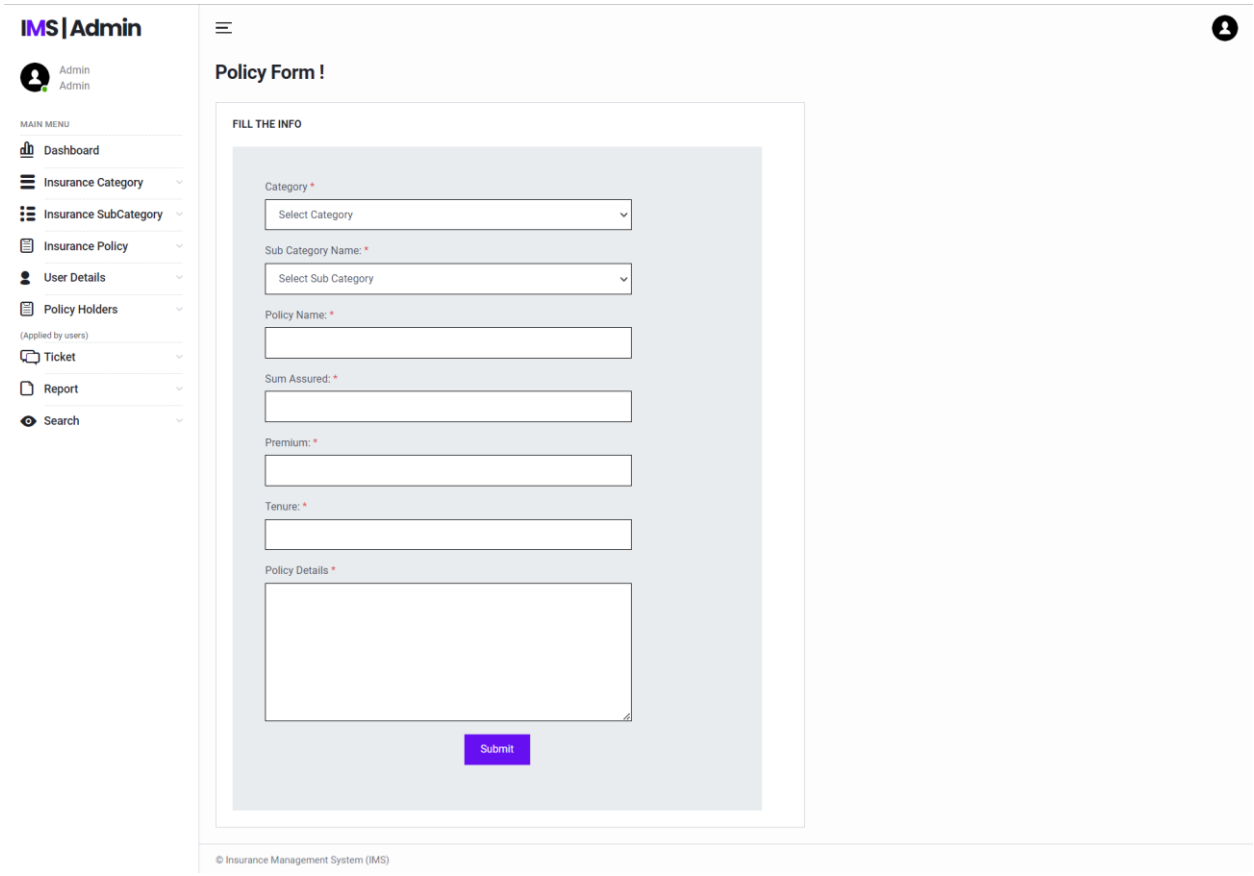
## Manage Sub Category



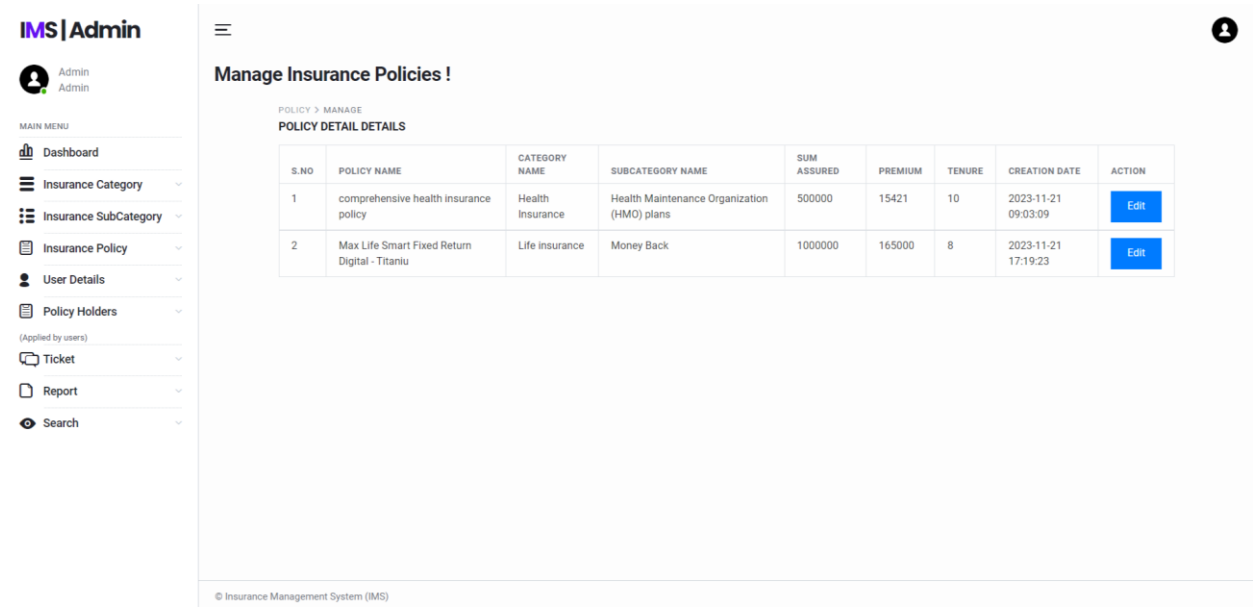

## Update Insurance Subcategory



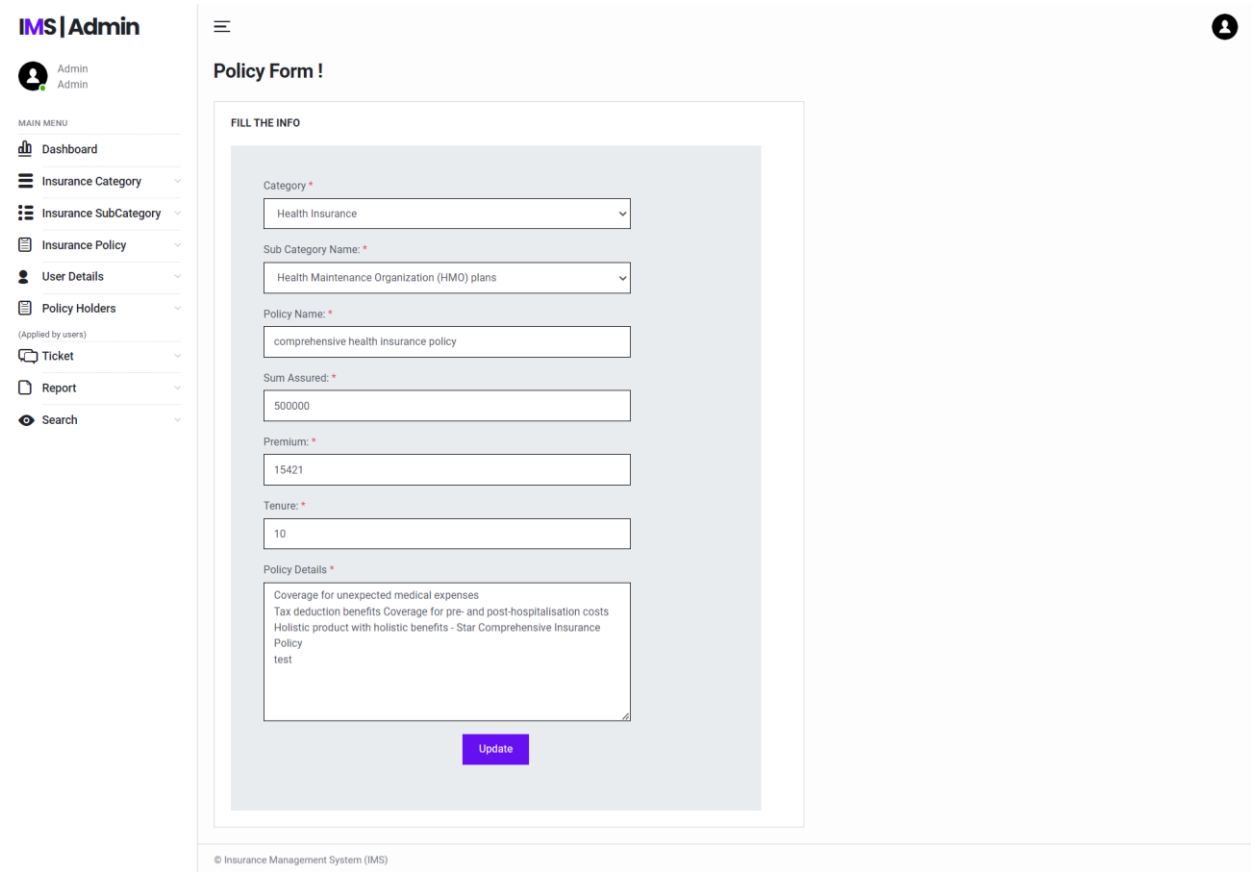
## Add Policy



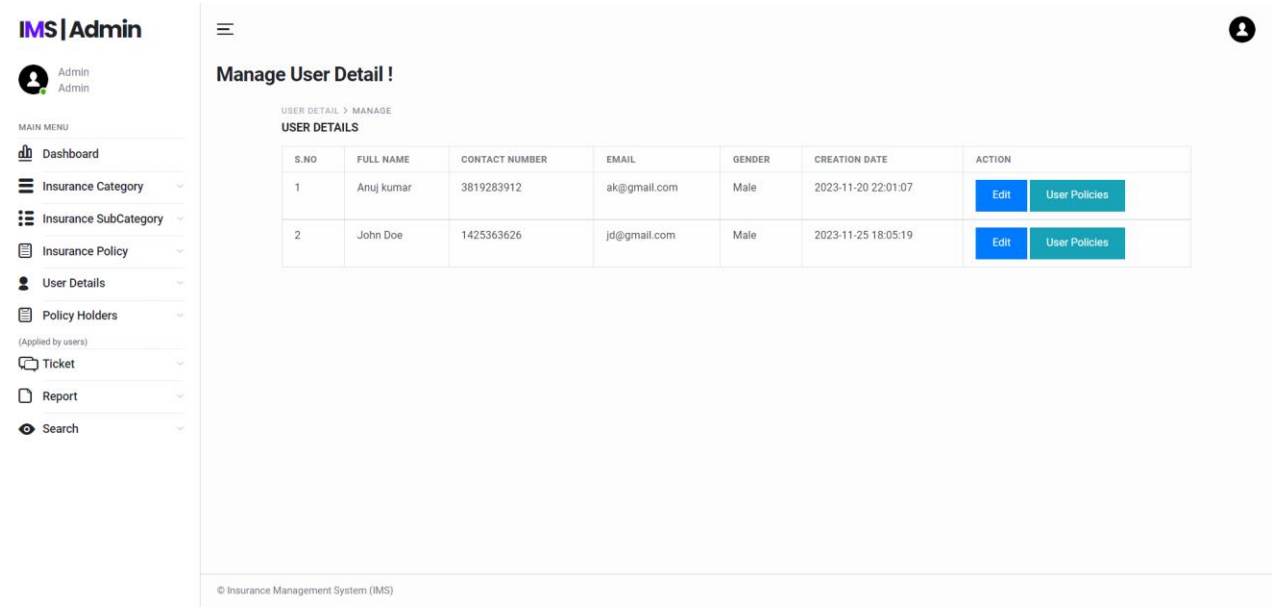

## Manage Policy



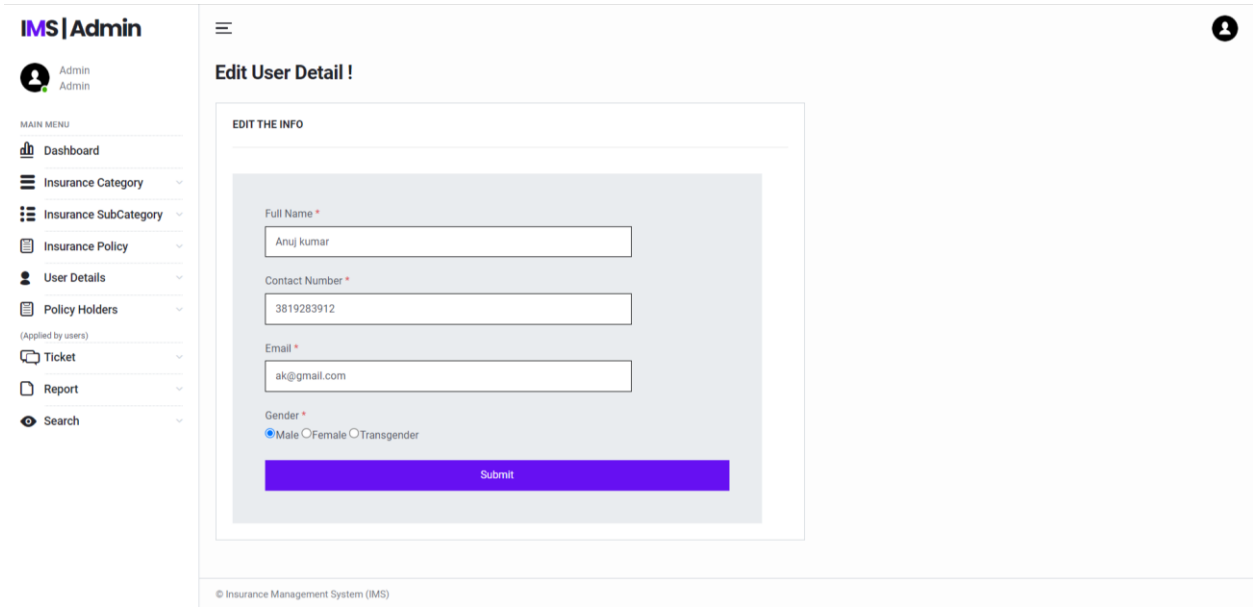
## Update Policy



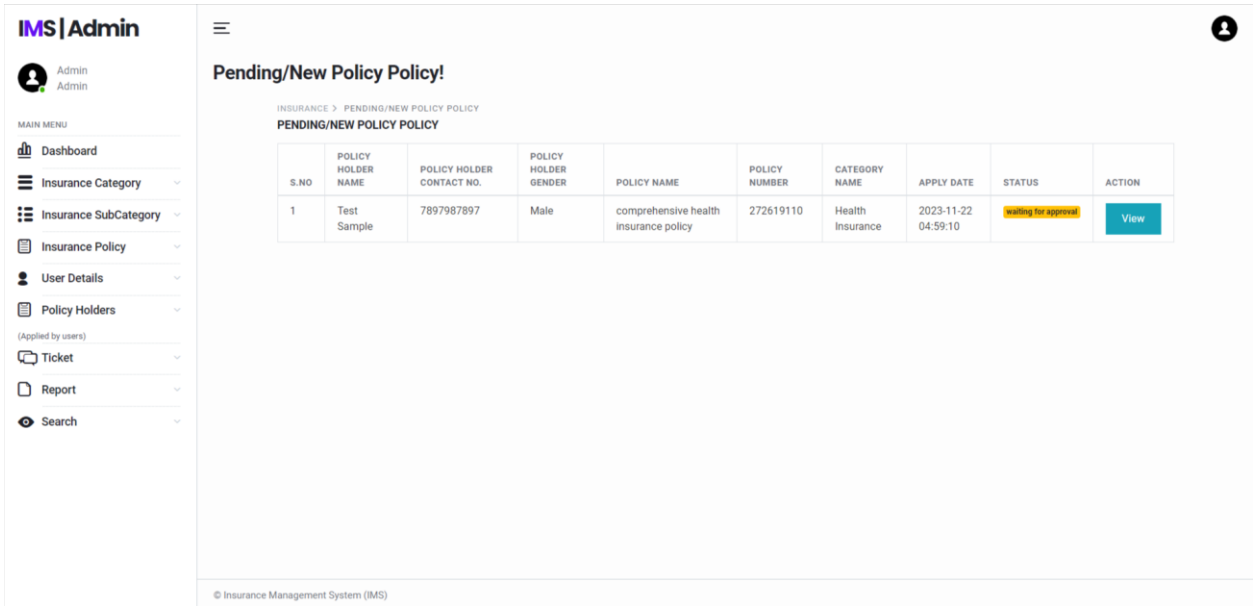

## User Details



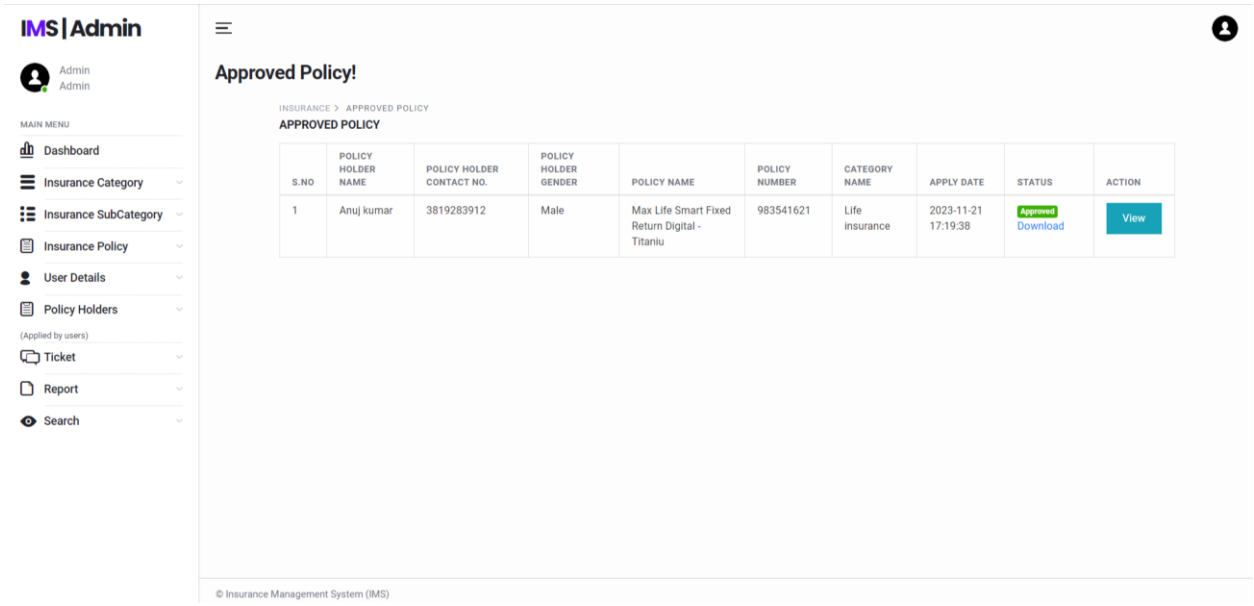
## Update User Details



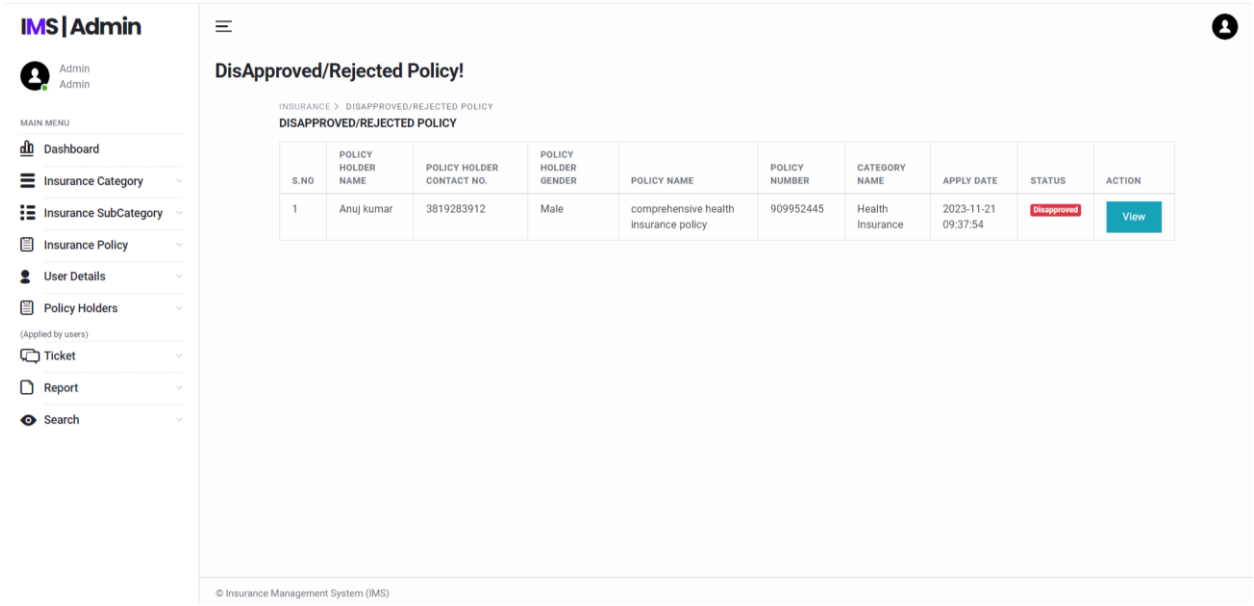

## Pending Policy Request



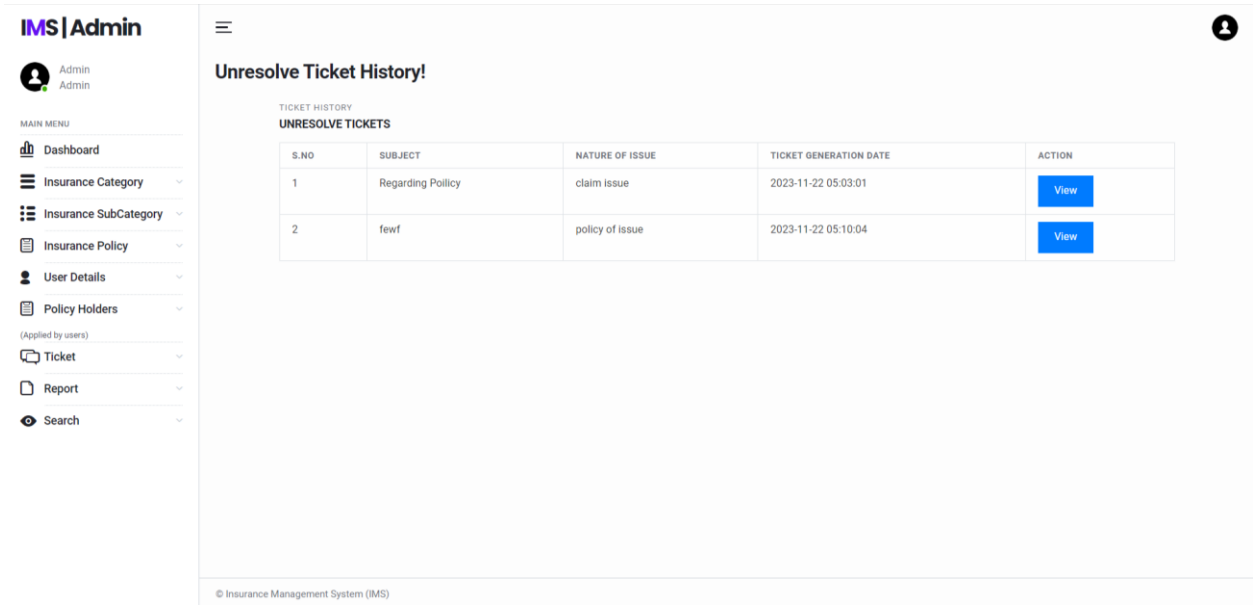
## Approved Policy



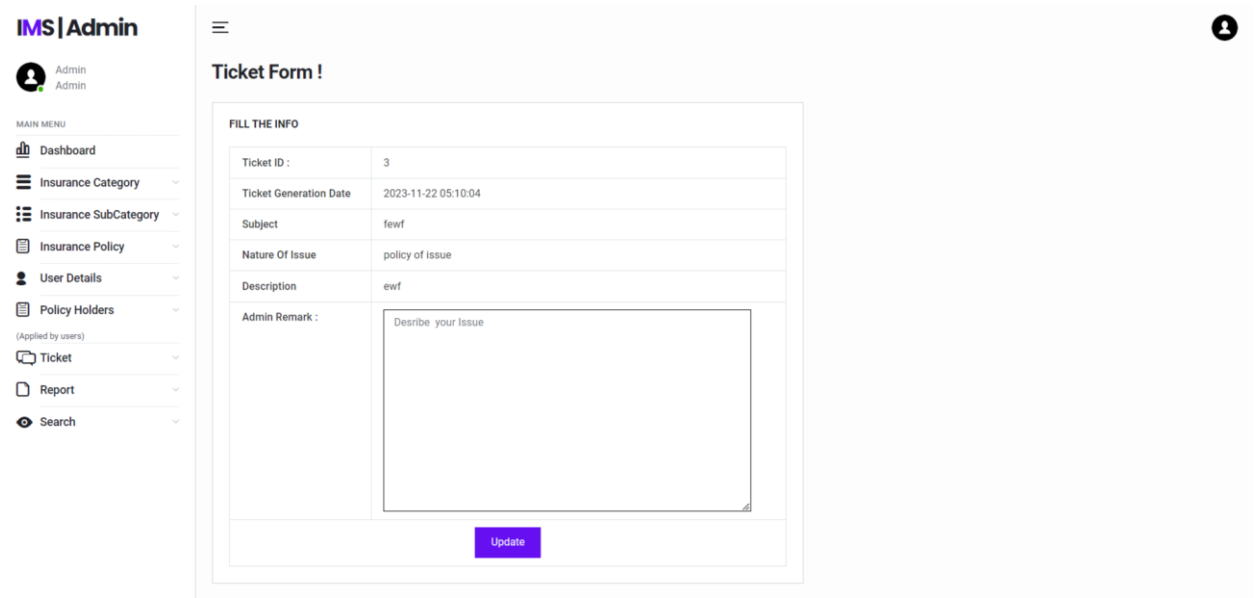

## Disapproved Policy



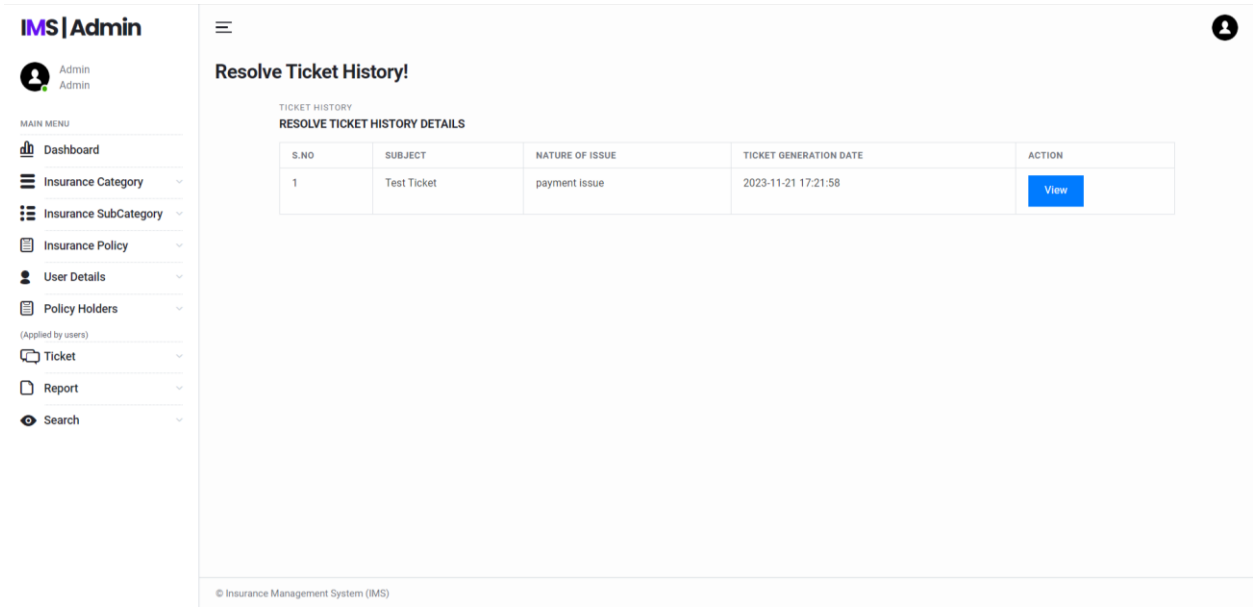
## Unresolved Ticket History



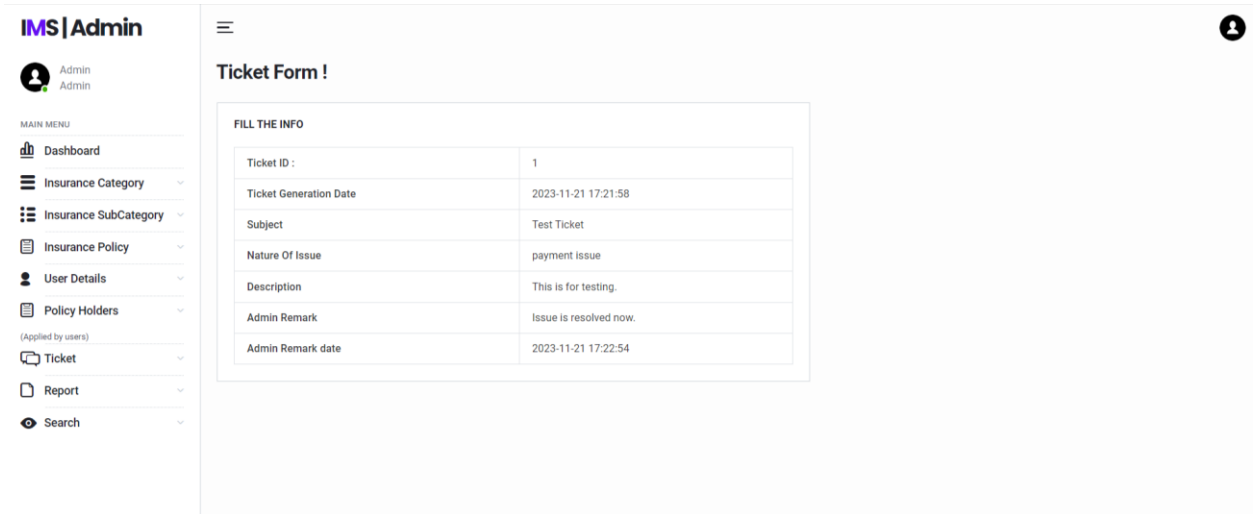

## View Unresolved Ticket



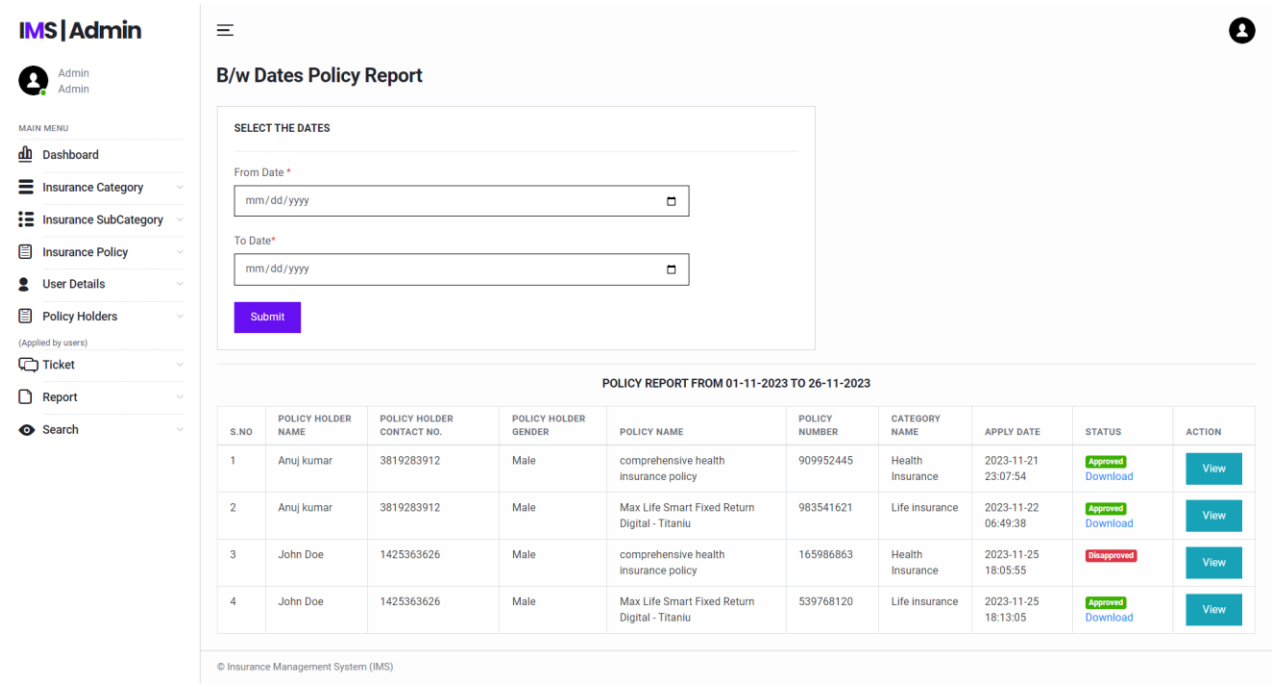
## Resolved Ticket



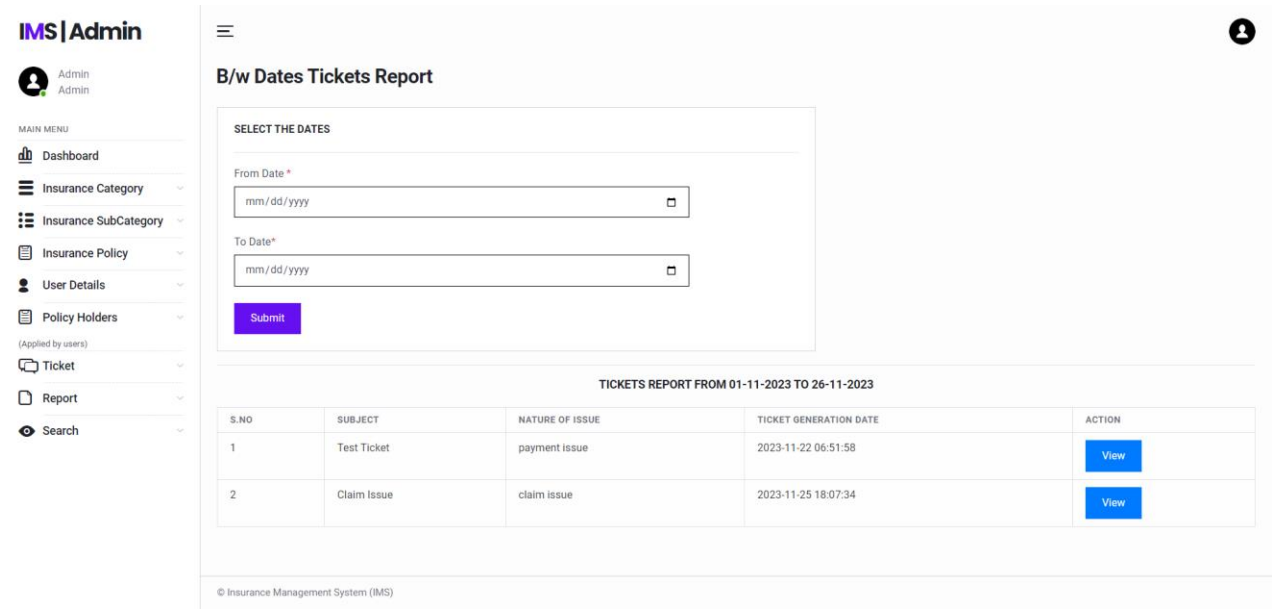

## View Resolved History



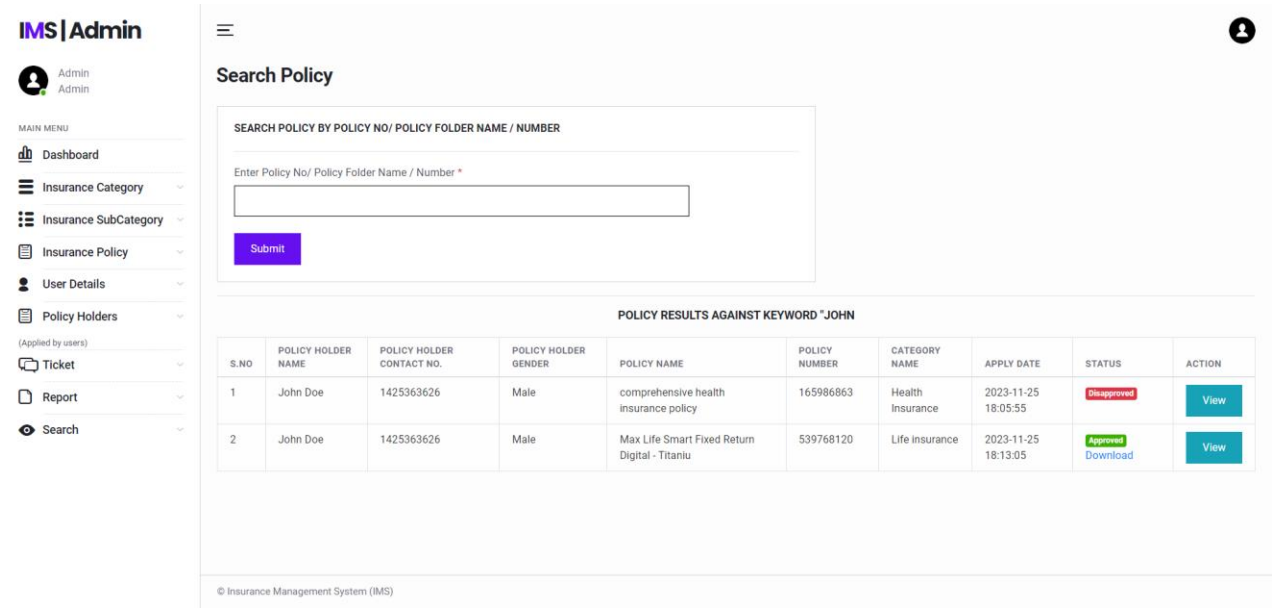
## Policy b/w Dates Report



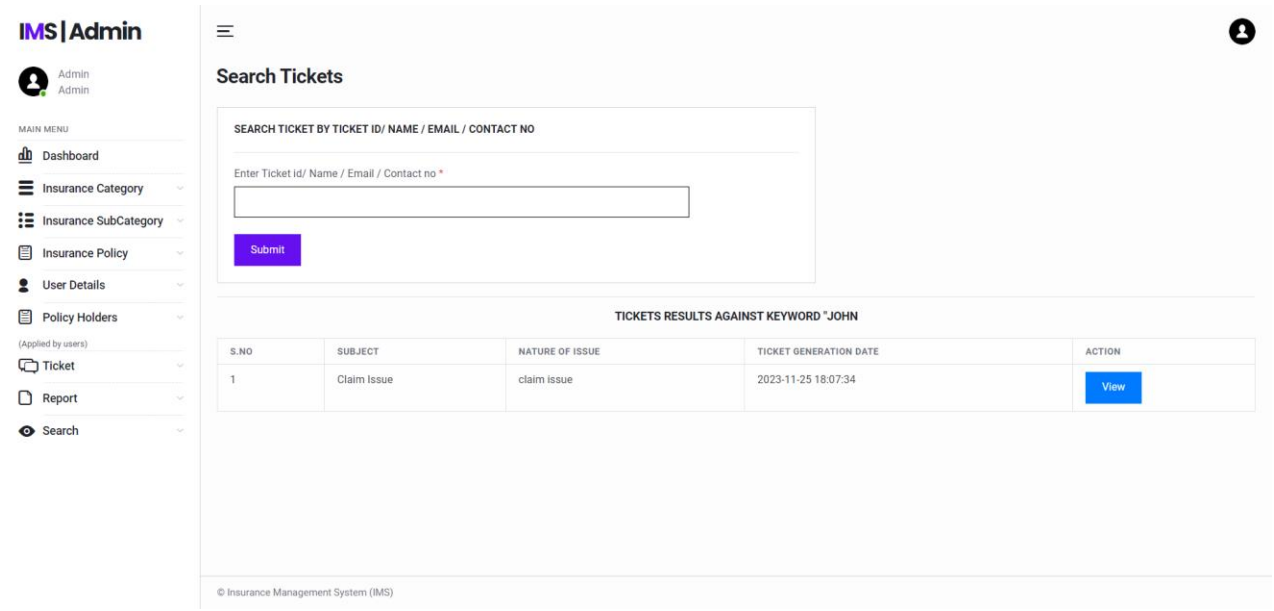

## B/w Date Ticket Report



## Search Policy

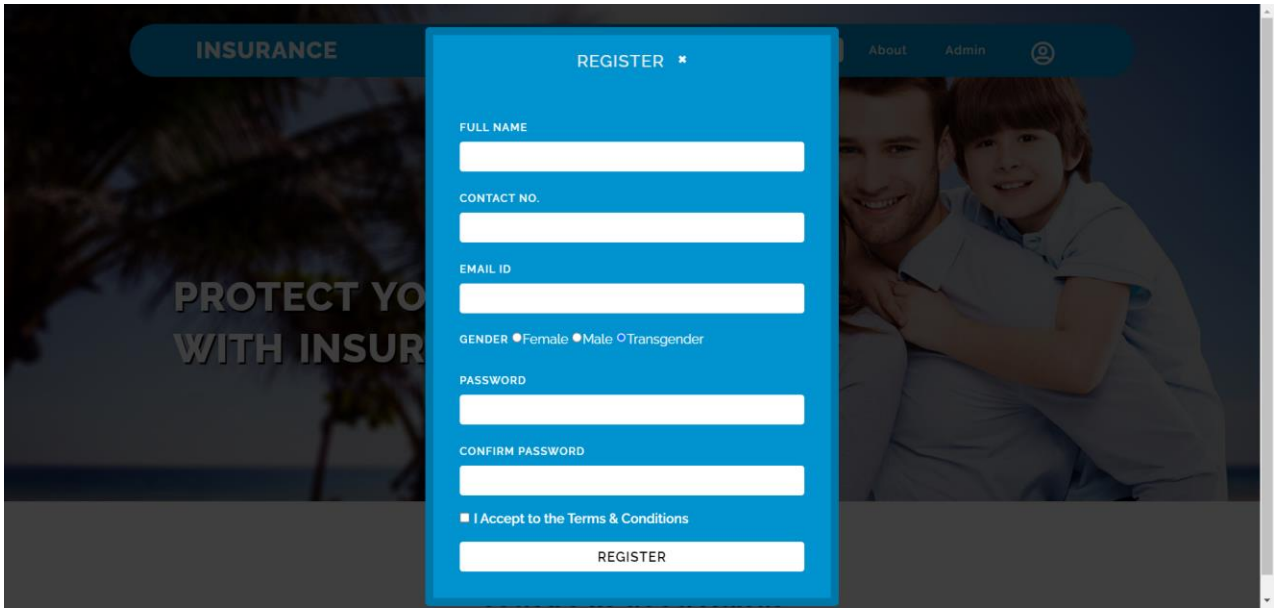



## Search Ticket

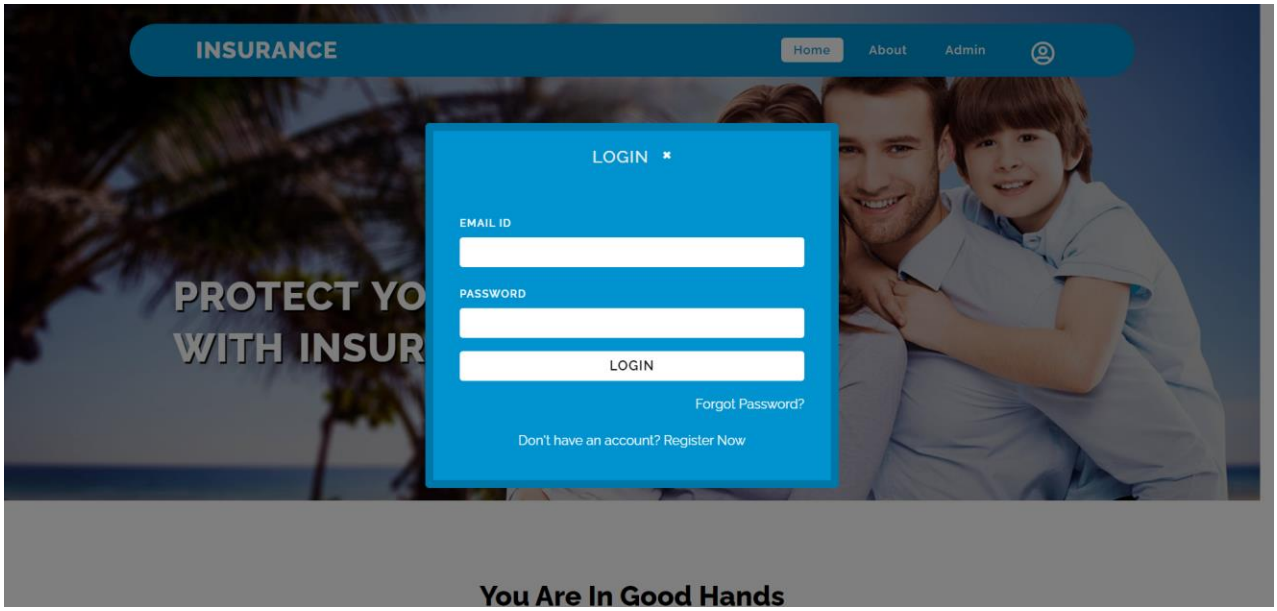


# User Panel

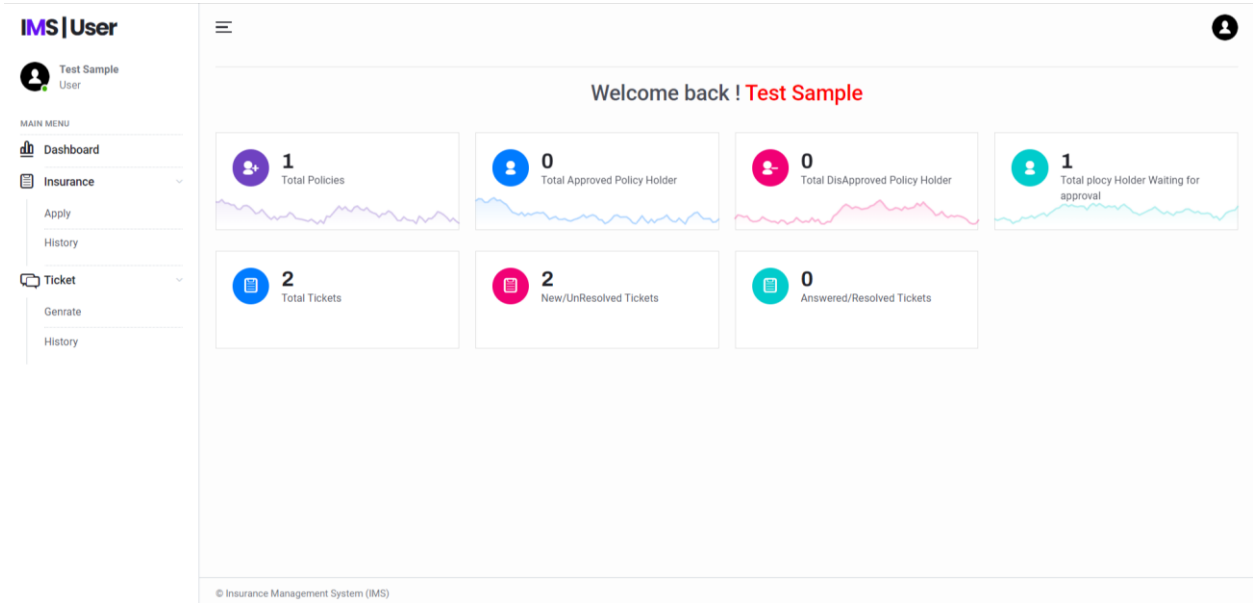
## Registration Page



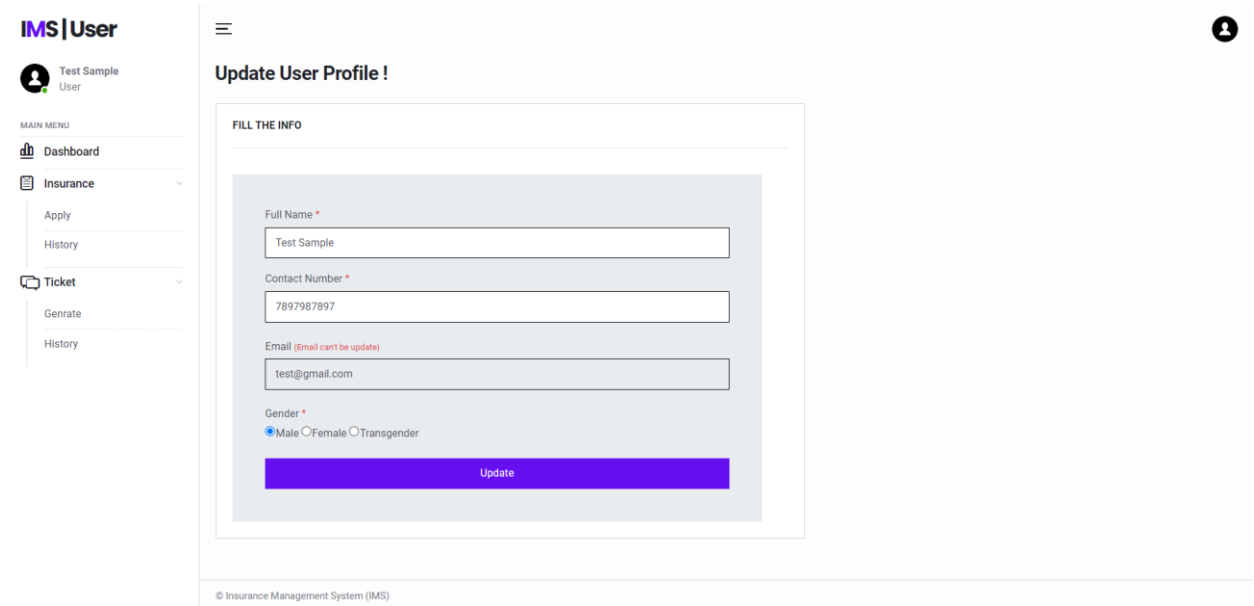

## Login Page



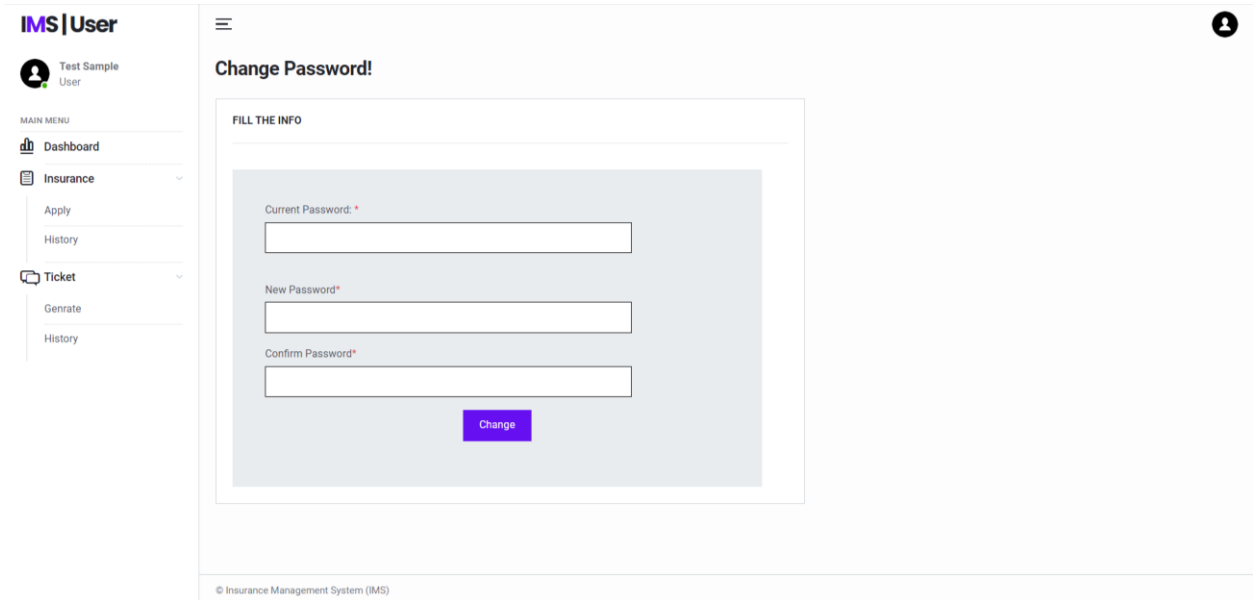
## Dashboard



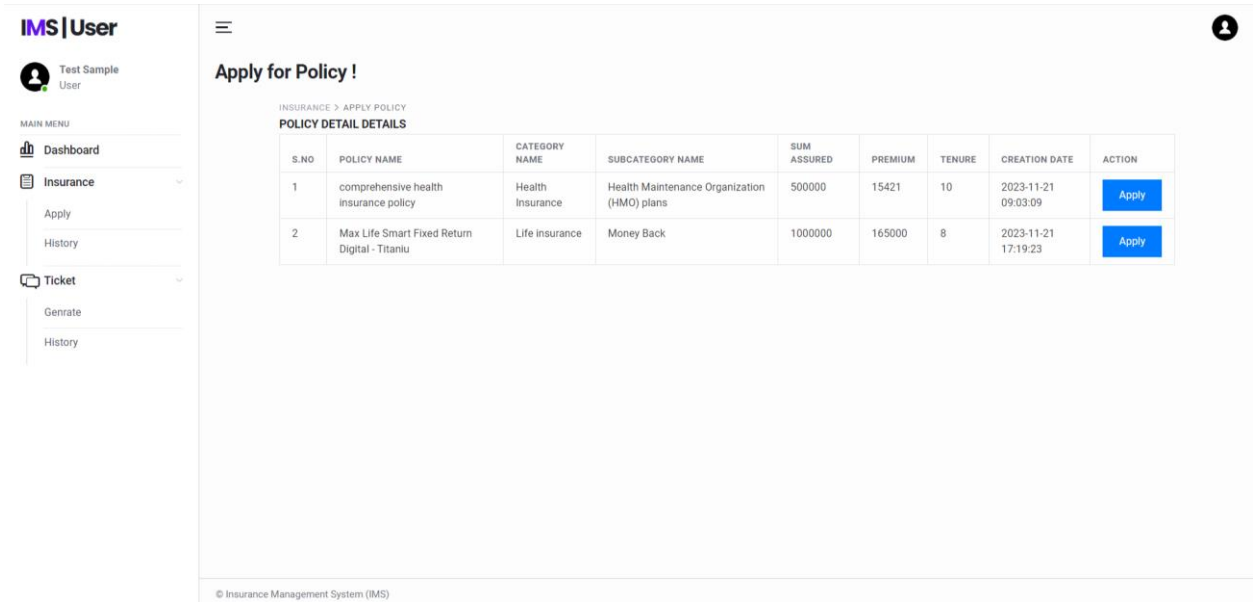

## Profile



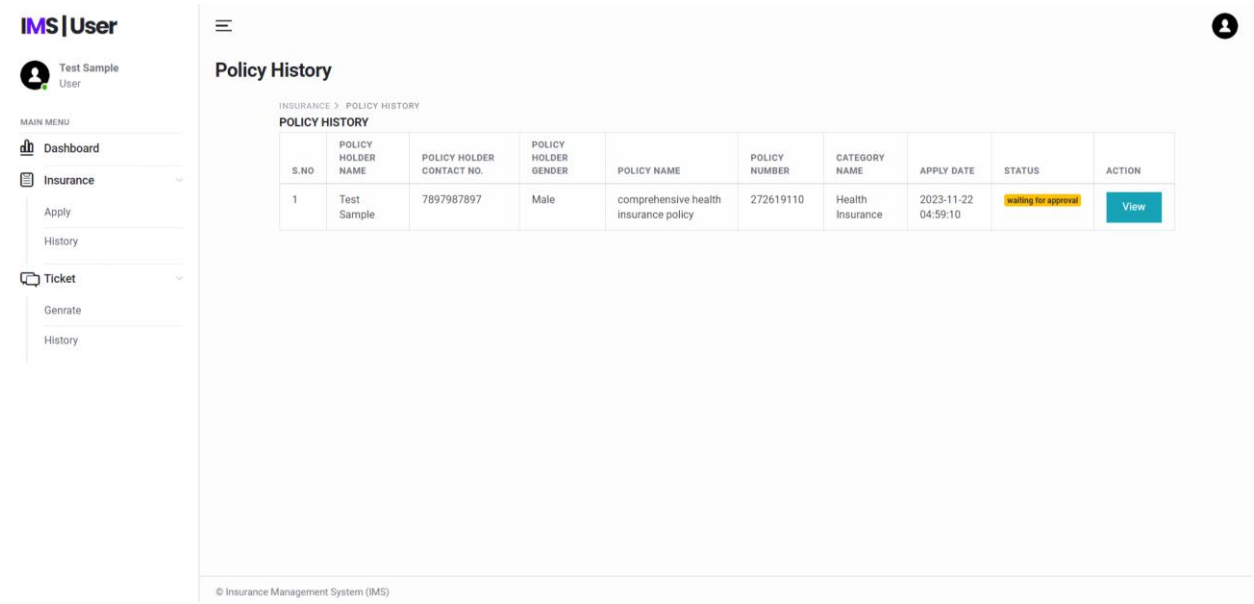
## Change Password



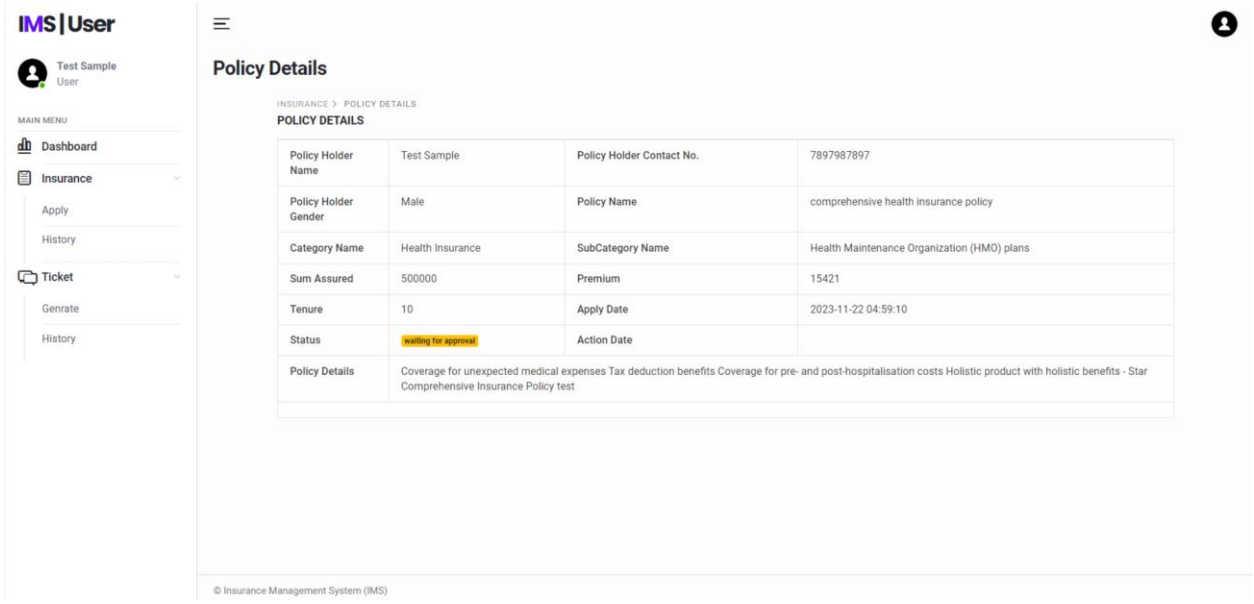

## Apply For Policy



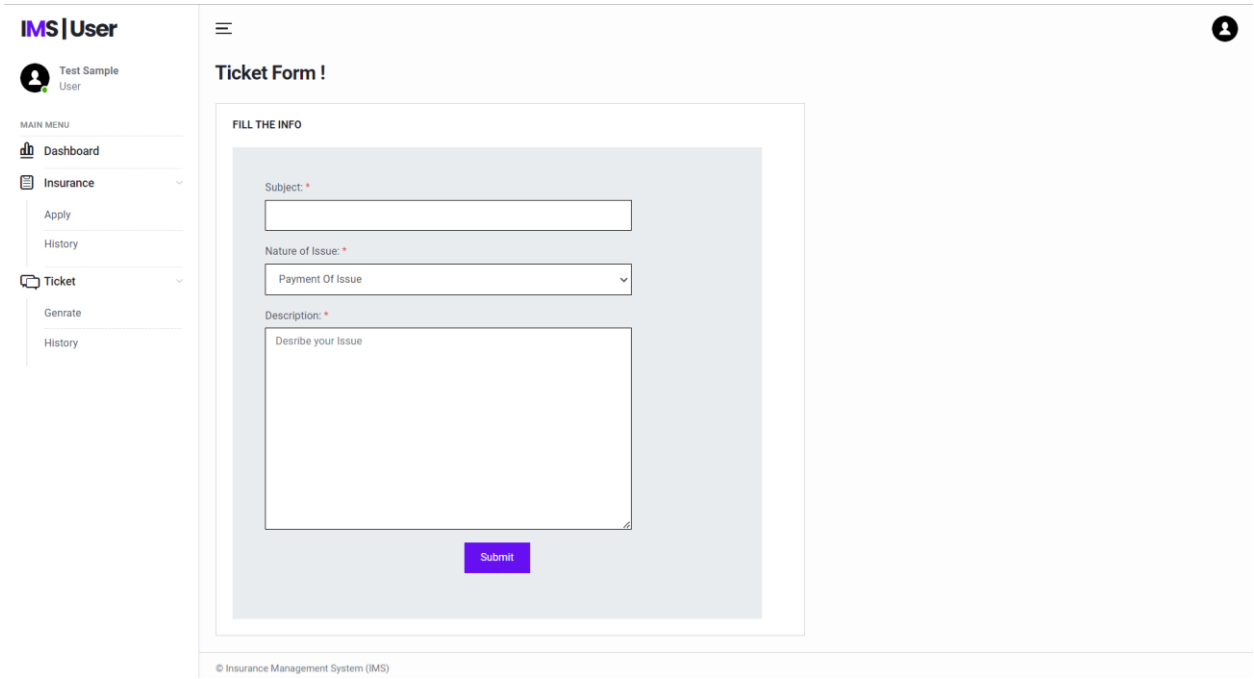
## Policy History



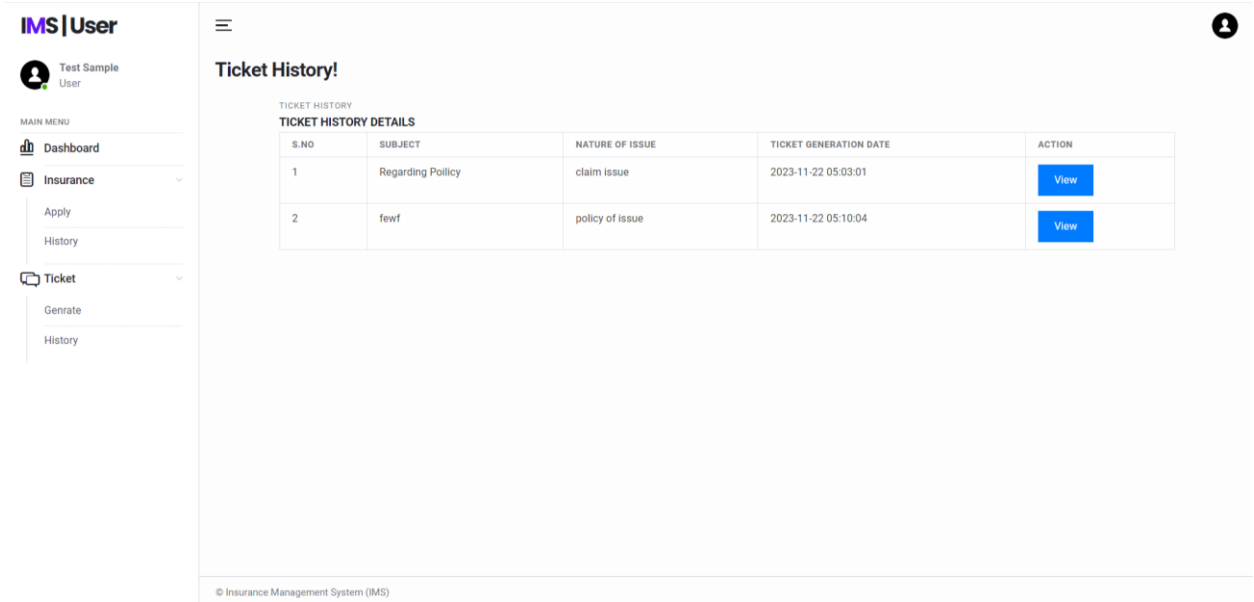

## Policy History Details



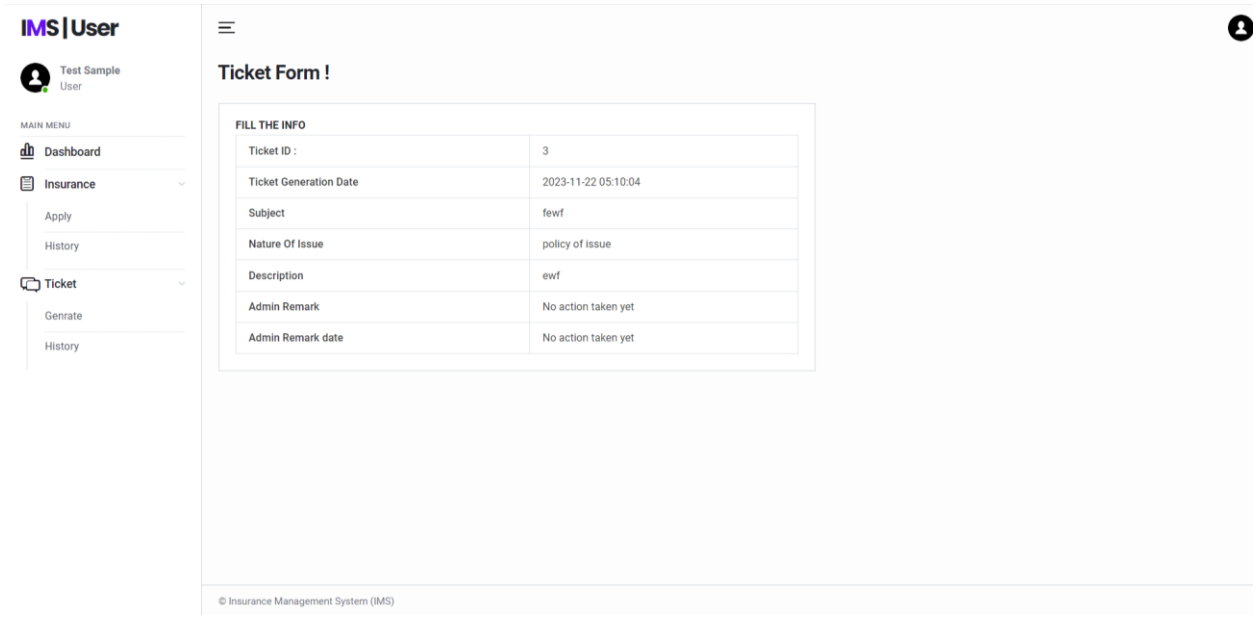
## Generate Ticket




## Ticket History



# View Details of Ticket



# CONCLUSION

This Application provides a computerized version of Insurance Management System which will benefit the company.

It makes entire process online and can generate reports. It has a facility of user login where user can apply for policy and admin can approve/disapprove the policy.

The Application was designed in such a way that future changes can be done easily. The following conclusions can be deduced from the development of the project.

- Automation of the entire system improves the productivity.
- It provides a friendly graphical user interface which proves to be better when compared to the existing system.
- It gives appropriate access to the authorized users depending on their permissions.
- It effectively overcomes the delay in communications.
- Updating of information becomes so easier.
- System security, data security and reliability are the striking features.
- The System has adequate scope for modification in future if it is necessary.

# BIBILIOGRAPHY


- ✓ www.w3schools.com
- ✓ *in.**php**.net*
- ✓ *en.wikipedia.org/wiki/**PHP***
- ✓ 4 . www.hotscripts.com/category/**php**/
- ✓ www.**apache**.org/
- ✓ www.**mysql**.com/click.php?e=35050